

Code Assessment of the fxswap v3 Smart Contracts

June 15, 2026

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	18
4	Terminology	19
5	Open Findings	20
6	Resolved Findings	25
7	Informational	57
8	Notes	79

1 Executive Summary

Dear Curve team,

Thank you for trusting us to help Curve with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of fxswap v3 according to [Scope](#) to support you in forming an opinion on their security risks.

The reviewed system is an updated version of Curve's TwoCrypto protocol. It introduces a new donation mechanism and adopts the StableSwap invariant in place of the CryptoSwap invariant. These changes are designed to enable more aggressive rebalancing using donated tokens and provide better swap price propositions for users.

The most critical subjects covered in our audit are functional correctness, asset security, and mathematical precision. Security regarding all the aforementioned subjects is good. The high severity issue and all medium severity issues have been addressed in the reviewed codebase.

Most severe is the possibility to extract donations from the pool, which was mitigated as described in the [Donations Can Be Extracted](#) issue. The corrected medium severity issues include [Donations can be directly absorbed by the pool](#), [Ramping can break the pool](#), and [Last part of ramping can be extracted](#).

The latest reviewed version also introduced policy integration issues. [Policy callbacks can be skipped by underfunding the capped call](#) allowed a caller to deliberately avoid synchronizing the external policy after a balanced withdrawal, and [Policy and pool timestamp drift on policy state updates](#) could make the pool and external policy contract reason from different elapsed-time assumptions. Both medium severity policy issues were corrected in the reviewed codebase. [Policy target prices can pass LP protection while mispricing the pool](#) was partially corrected by bounding policy-provided targets, but the policy contract should still be treated as part of the system's trust boundary and operational risk model.

Additional areas evaluated include fee computation, correct integration of StableSwapMath, policy-controlled rebalancing, ramping behavior, and specification adherence. Security regarding these subjects is good. Most low severity issues were corrected, including [calc_token_amount applies withdrawal fees in the wrong direction](#) and [Ramping pool can become stuck at the imbalance boundary](#). The remaining low severity issues are either risk accepted or partially corrected. These include the accepted risks around [Donations can be extracted with swaps](#), [Griefing attack on liquidity additions](#), and [Parameters check mismatch](#), as well as the partially corrected issues [Admin is able to DoS the system](#) and [Policy target prices can pass LP protection while mispricing the pool](#).

The overall security of the system heavily depends on the configuration and use, especially as there are multiple smaller acknowledged or risk-accepted issues that, in most conditions, are irrelevant to the normal operation of the pool but can become relevant in edge cases or when combined with other issues.

In summary, we would rate the security as good if the remaining recommendations are followed and the pool is monitored and managed accordingly.

Given the extensive complexity of the system, we recommend that the Curve team conduct comprehensive testing, including fuzzing and stateful testing, to ensure the system's robustness.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity



1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	1
• Code Corrected	1
Medium -Severity Findings	7
• Code Corrected	7
Low -Severity Findings	17
• Code Corrected	12
• Code Partially Corrected	2
• Risk Accepted	3

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the fxswap v3 repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	22 May 2025	3a15bf74f0b44b55817e541799b73641e6f8a849	Initial Version
2	17 June 2025	a9844719d794e83cc160dfe9bb088a73f73ef5be	Fixes
3	19 June 2025	dbbbfffe9133665060c9a79d99b414bb4c4cc593	Fixes
4	04 July 2025	c58b667e9c7217be3a756daf420c47b6cb26ffaf	Fixes
5	11 July 2025	c2f5a6b55fe49c9d40226e5c70c6ded8fea93ed4	Fixes
6	04 August 2025	66a3178e6c8713d624c64ab3e3cfc9edc1d9f8e9	Fixes
7	09 September 2025	52bddd91b211abd65ed7e2aa5814d4084f9018ff	fxswap v1
8	29 October 2025	f601f641906b1c55078c421ad8057043e20d7602	Fixes
9	04 May 2026	6996a9495d753017371184f82a67e94fb51711d2	Fixes
10	11 May 2026	2c645ca604a4a0878e08f2f1581e5c4ae1c8f8d4	Fixes

For the Vyper smart contracts, the compiler version 0.4.1 was initially chosen. In the latest commit at the time of the assessment, the version 0.4.3 and the EVM version prague were used.

The assessment was performed on the following contracts:

- `contracts/main/Twocrypto.vy`
- `contracts/main/TwocryptoFactory.vy`
- `contracts/main/TwocryptoView.vy`
- `contracts/main/StableswapMath.vy`

As of Version 9, the following contracts was added to the scope of the assessment:

- `contracts/main/TwocryptoPolicy.vy`

2.1.1 Excluded from scope

Anything not explicitly mentioned in the scope section is excluded from the assessment. This includes, but is not limited to:

- `contracts/main/LiquidityGauge.vy`
- `contracts/old/`

2.2 System Overview

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

The new version of Curve's Twocrypto contracts introduces a new functionality to donate funds to a pool and increase the chance of successful price tweaks. Additionally, the internal logic when these rebalancing operations happen is changed to mitigate some shortcomings of the previous versions. With a more aggressive and reliable rebalancing mechanism, the pool switched to the simpler Stableswap invariant. Other changes try to enhance code quality and introduce minor improvements.

2.2.1 Architecture Overview

The system consists of the following main components:

1. **TwoCrypto Pool**: The core AMM contract implementing user functions, liquidity management, fee structure, and rebalancing mechanisms
2. **StableswapMath**: Mathematical library for invariant operations
3. **TwoCrypto Factory**: Factory contract for deploying pools and gauges
4. **TwoCrypto View**: Read-only contract for querying pool state and previewing operations

2.2.2 TwoCrypto Pool

The TwoCrypto pool is a hybrid AMM that combines constant product and constant sum invariants for two unpegged assets. When interacting with the pool, users can perform various actions that affect the pool's state and invariant.

2.2.2.1 Stableswap Invariant

The pool implements a hybrid invariant defined in the "xp-space" (balances scaled by price_scale):

$$An^n \sum xp_i + D = ADn^n + \frac{D^{n+1}}{n^n \prod xp_i}$$

Where:

- A is the amplification coefficient
- n is the number of coins
- xp_i is the value of the i -th coin (amount times price in respect to coin 0)
- D is the invariant (i.e. the total virtual balance when all coins are equal)

From this invariant, there are two fundamental operations:

Solving for x_j (Token Exchange):

Calculate the amount of tokens to receive for a given amount of tokens in a swap (get_y).

We derive Newton's iteration to solve $x p_j$, given $\{x p_i\}_{i \neq j}$, A , and D .
Start with the invariant:

$$A n^n \sum x p_i + D = A D n^n + \frac{D^{n+1}}{n^n \prod x p_i}$$

Isolate $x p_j$:

$$A n^n \left(x p_j + \sum_{i \neq j} x p_i \right) + D = A D n^n + \frac{D^{n+1}}{n^n (x p_j \prod_{i \neq j} x p_i)}$$

Multiply both sides by $x p_j$:

$$A n^n \left(x p_j^2 + x p_j \sum_{i \neq j} x p_i \right) + D x p_j = A D n^n x p_j + \frac{D^{n+1}}{n^n \prod_{i \neq j} x p_i}$$

Divide by $A n^n$:

$$x p_j^2 + x p_j \sum_{i \neq j} x p_i + \frac{D}{A n^n} x p_j = D x p_j + \frac{D^{n+1}}{A n^{2n} \prod_{i \neq j} x p_i}$$

Bring terms to one side:

$$x p_j^2 + x p_j \left(\sum_{i \neq j} x p_i + \frac{D}{A n^n} - D \right) - \frac{D^{n+1}}{A n^{2n} \prod_{i \neq j} x p_i} = 0$$

Define:

$$f(x p_j) = x p_j^2 + x p_j \left(\sum_{i \neq j} x p_i + \frac{D}{A n^n} - D \right) - \frac{D^{n+1}}{A n^{2n} \prod_{i \neq j} x p_i}$$

Taking the derivative:

$$f'(x p_j) = 2 x p_j + \sum_{i \neq j} x p_i + \frac{D}{A n^n} - D$$

Given the Newton's iteration

$$x p_j^{\text{new}} = x p_j - \frac{f(x p_j)}{f'(x p_j)} = \frac{x p_j f'(x p_j) - f(x p_j)}{f'(x p_j)}$$

Substitute in the expressions:

$$\begin{aligned}
xp_j^{\text{new}} &= \frac{xp_j \left(2xp_j + \sum_{i \neq j} xp_i + \frac{D}{An^n} - D \right) - \left(xp_j^2 + xp_j \left(\sum_{i \neq j} xp_i + \frac{D}{An^n} - D \right) - \frac{D^{n+1}}{An^{2n} \prod_{i \neq j} xp_i} \right)}{2xp_j + \sum_{i \neq j} xp_i + \frac{D}{An^n} - D} \\
&= \frac{xp_j^2 + \frac{D^{n+1}}{An^{2n} \prod_{i \neq j} xp_i}}{2xp_j + \sum_{i \neq j} xp_i + \frac{D}{An^n} - D} \\
&= \frac{xp_j^2 + c}{2xp_j + b - D}
\end{aligned}$$

where:

$$\begin{aligned}
S &= \sum_{i \neq j} xp_i \\
b &= S + \frac{D}{An^n} \\
c &= \frac{D^{n+1}}{An^n \cdot n \cdot n^{n-1} \cdot \prod_{i \neq j} xp_i}
\end{aligned}$$

Finally, convert the value xp_i to balance x_i .

Solving for D (Deposits, fee accrual):

Find a new D (newton_D) when either the balances, price scale, A or gamma changes:

We derive Newton's iteration for solving D given $\{xp_i\}_{i=1}^n$ and A .

$$f(D) = An^n \sum xp_i + D(1 - An^n) - \frac{D^{n+1}}{n^n \prod xp_i} = 0$$

Or equivalently:

$$f(D) = \frac{D^{n+1}}{n^n \prod xp_i} - D(1 - An^n) - An^n \sum xp_i$$

Taking the derivative:

$$f'(D) = \frac{(n+1)D^n}{n^n \prod xp_i} + An^n - 1$$

Using Newton's formula:

$$D_{\text{new}} = D - \frac{f(D)}{f'(D)} = \frac{Df'(D) - f(D)}{f'(D)}$$

Substituting $f(D)$ and $f'(D)$:

$$\begin{aligned}
D_{\text{new}} &= \frac{D \left(\frac{(n+1)D^n}{n^n \prod x p_i} + A n^n - 1 \right) - \left(\frac{D^{n+1}}{n^n \prod x p_i} - D(1 - A n^n) - A n^n \sum x p_i \right)}{\frac{(n+1)D^n}{n^n \prod x p_i} + A n^n - 1} \\
&= \frac{\frac{(n+1)D^{n+1}}{n^n \prod x p_i} + A n^n D - D - \frac{D^{n+1}}{n^n \prod x p_i} + D - A n^n D + A n^n \sum x p_i}{\frac{(n+1)D^n}{n^n \prod x p_i} + A n^n - 1} \\
&= \frac{\frac{n D^{n+1}}{n^n \prod x p_i} + A n^n \sum x p_i}{\frac{(n+1)D^n}{n^n \prod x p_i} + A n^n - 1} \\
&= \frac{D(N \cdot \text{dp} + \text{ann} \cdot S)}{(n+1) \cdot \text{dp} + D(\text{ann} - 1)}
\end{aligned}$$

where:

$$\begin{aligned}
\text{ann} &= A n^n \\
S &= \sum x p_i \\
\text{dp} &= \frac{D^{n+1}}{n^n \prod x p_i}
\end{aligned}$$

2.2.2.2 Price Oracle, Ramping and Rebalancing Mechanism

Price Oracle:

The price oracle uses an exponential moving average (EMA) of the pool's last prices to track market conditions:

$$\alpha = e^{-\Delta t / \tau},$$

$$P_{\text{oracle,new}} = (1 - \alpha) \min(P_{\text{spot}}, 2P_{\text{scale}}) + \alpha P_{\text{oracle,old}}.$$

With:

P_{spot}	Spot price of the last trade ("last_prices")
P_{scale}	Current band centre ("price_scale")
$P_{\text{oracle,old}}$	Stored oracle price from previous block
$P_{\text{oracle,new}}$	Oracle price after the update
Δt	Elapsed time ($t - t_{\text{last}}$)
τ	EMA time constant (<code>ma_time</code>)
α	Decay factor $e^{-\Delta t / \tau}$

Price Scale Adjustments:

Price tweaks are crucial for keeping the pool competitive. When conditions are met, the price scale adjusts toward the oracle price:

$$p_{\text{scale}}^{t+1} = \frac{p_{\text{scale}}^t (\text{norm} - \text{adj}) + (\text{adj} \cdot p_{\text{oracle}})}{\text{norm}}$$

$$\text{norm} = \left| \frac{p_{\text{oracle}}}{p_{\text{scale}}^t} - 1 \right|$$

$$\text{adj} = \begin{cases} \frac{\text{norm}}{5}, & \text{if } \frac{\text{norm}}{5} > \text{rebalancing_params}[1], \\ \text{rebalancing_params}[1], & \text{otherwise.} \end{cases}$$

The `tweak_price` function is invoked in nearly all operations except for balanced liquidity withdrawals. Price scale updates shift the linear portion of the curve to the new price scale, preserving swap stability and pool competitiveness.

A price scale adjustment is triggered only if all the following conditions are met (note, the conditions changed in later versions):

Given $\text{threshold_vp} = 1 + (\text{xcp_profit} - 1)/2$:

1. $\text{virtual_price} > \text{threshold_vp} + \text{allowed_extra_profit}$
Ensures that half of the profit growth is reserved for LPs and the admin, while the remainder is used for price scale tweaks.
2. $\text{norm} > \text{adjustment_step}$
Ensures that the price scale is only adjusted when the difference between the current and target price scale is significant enough.
3. $\text{new_virtual_price} \geq \text{threshold_vp}$
Ensures that after the adjustment, half of the profit growth remains reserved for LPs and the admin.

Donations can be used to help satisfy conditions 1 and 3 by "boosting" the virtual price. Boosting means burning shares from the donation pool to increase the virtual price, defined as $\text{xcp}/\text{total_supply}$. The boosted virtual price is $\text{xcp}/(\text{total_supply} - \text{donation_shares_available})$. Thus, the first condition becomes:

- $\text{boosted_virtual_price} > \text{threshold_vp} + \text{allowed_extra_profit}$ This ensures the boosted virtual price exceeds the threshold, permitting a price scale adjustment.

For the third condition, instead of $\text{new_virtual_price} = \text{new_xcp} / \text{total_supply}$, we use $\text{new_virtual_price} = \text{new_xcp} // (\text{total_supply} - \text{donation_shares_to_burn})$, where $\text{donation_shares_to_burn}$ is the amount needed to reach exactly threshold_vp . The third condition otherwise remains unchanged.

Parameter Changes:

The admin of the pool can change the parameters A and gamma using the `ramp_A_gamma` function. Similarly, the various other parameters can be tweaked using `apply_new_parameters`.

Protection Against Exploitation:

Changes in D caused by changing A or price_scale modifications can be exploited to extract value. Therefore:

- **A Parameter:** Uses a "ramping" mechanism to change values linearly over time, preventing exploitation of curve updates.
- **Price Scale:** Follows the price oracle with small, controlled adjustments to avoid too large changes that could be exploited.

2.2.2.3 User Operations

The pool supports three main user actions:

1. **Liquidity Provision:** Users can add liquidity to the pool, either as regular liquidity providers or as donors.
2. **Liquidity Withdrawal:** Users can withdraw their liquidity, either in a balanced or unbalanced manner.
3. **Token Exchange:** Users can swap tokens within the pool, either by sending tokens first or allowing the pool to transfer them.

2.2.2.3.1 Liquidity Provision

Adding Liquidity:



Users can add liquidity through the `add_liquidity` function, supporting both regular provision and donations (`is_donation=True`).

The user's contribution is defined as the delta in D. LP tokens are calculated as:

$$d_{\text{token}} = \begin{cases} \text{token_supply} \left(\frac{D}{D_{\text{old}}} - 1 \right), & D_{\text{old}} > 0, \\ \frac{D}{2\sqrt{\text{price_scale}}}, & D_{\text{old}} = 0. \end{cases}$$
$$\text{lp_minted} = d_{\text{token}} \cdot N = 2$$

Donations:

In case of a donation, the contribution is not minted but added to the total supply. Donations are tracked separately in `donation_shares` and made available linearly over time, making funds available for burning through rebalancing.

2.2.2.3.2 Liquidity Withdrawal

Balanced Withdrawal:

Using `remove_liquidity`, users receive tokens proportional to their LP token share without fees. The function is intentionally designed to be simple and not dependent on invariant calculation, ensuring a safe exit for liquidity providers in case of an issue with the pool.

Unbalanced Withdrawal:

Given a fixed amount of one token to be withdrawn (0 for `remove_liquidity_one_coin`, or a specific amount for `remove_liquidity_fixed_out`), the process follows these steps:

1. Calculate user's share of D (dD) relative to LP tokens:

$$dB = \frac{\text{user_LP} \cdot D}{\text{token_supply}}$$

2. Calculate token amounts using `get_y` with updated D (D - dD)
3. Apply fees on the new token balances
4. Recalculate `get_y` with the updated balances (considering the fee) and updated D (D - dD).

2.2.2.3.3 Token Exchange

Swap Functions:

- `exchange`: Requires token allowance for transfer
- `exchange_received`: Uses optimistic transfers (tokens sent first)

Both functions call the internal `_exchange` function, which:

1. Calculates output amount using `get_y`
2. Applies fees
3. Updates oracle price
4. Attempts price scale tweaks
5. Transfers tokens to user

2.2.2.3.4 Fee Structure

Fees are applied to most operations, with higher fees for operations that create more imbalance:

$$b = n^n \frac{\prod x_i}{(\sum x_i)^n},$$

$$g = \frac{\gamma b}{\gamma b + 1 - b},$$

$$\text{base_fee} = g \text{ fee}_{\text{mid}} + (1 - g) \text{ fee}_{\text{out}}.$$

$$\text{total_fee} = \frac{\text{base_fee}}{2} \frac{|\text{amount}_1 - \bar{a}| + |\text{amount}_2 - \bar{a}|}{\text{amount}_1 + \text{amount}_2} + \text{noise_fee},$$

$$\bar{a} = \frac{1}{2}(\text{amount}_1 + \text{amount}_2).$$

Fee Types:

- **Swap Fees:** `base_fee`
- **Adding and Removing liquidity:** `total_fee`
- **Donation Fee:** No fees except noise fee for donations

2.2.2.3.5 Administrative Functions

- `ramp_A_gamma`: Start ramping A and gamma parameters (gamma is unused in versions before **Version 9**). From **Version 9** onward, a magic gamma value indicates whether post-deploy initialization is enabled)
- `stop_ramp_A_gamma`: Stop parameter ramping
- `apply_new_parameters`: Apply new pool parameters
- `set_donation_parameters`: Configure donation release and protection parameters
- `set_fee_parameters`: Configure the reserved profit fraction and admin fee split
- `set_policy_contract`: Configure an optional external policy contract
- `change_allowlist`: Configure the optional LP allowlist

2.2.3 TwoCrypto View

The view contract provides gas-optimized, read-only functions for off-chain applications:

Preview Functions:

- `get_dy`: Preview swap output amounts
- `get_dx`: Calculate required input for desired output
- `calc_withdraw_one_coin`: Preview single-token withdrawals
- `calc_fee_withdraw_one_coin`: Calculate withdrawal fees
- `calc_fee_get_dy`: Calculate swap fees
- `calc_token_amount`: Preview LP token amounts for liquidity operations
- `calc_fee_token_amount`: Calculate fees for liquidity operations

2.2.4 TwoCrypto Factory

The factory contract enables permissionless deployment of pools and gauges with specified parameters.

Deploying a Pool

The `deploy_pool()` function allows anyone to create pools by providing:

- Pool name and symbol
- Two tokens for the pool
- Implementation ID (set by admin via `set_pool_implementation()`)
- Pool parameters: `A`, `gamma`, `mid_fee`, `out_fee`, `fee_gamma`, `allowed_extra_profit`, `adjustment_step`, `ma_exp_time`, `initial_price`

Multiple pools can be deployed with identical tokens and parameters.

Deploying a Gauge

The `deploy_gauge()` function creates gauge contracts for existing pools. Only one gauge per pool is permitted.

Admin-Restricted Functions:

- `set_fee_receiver()`: Configure fee collection address
- `set_pool_implementation()`: Update pool implementation
- `set_gauge_implementation()`: Update gauge implementation
- `set_views_implementation()`: Update view implementation
- `set_math_implementation()`: Update math implementation
- `commit_transfer_ownership()`: Transfer factory ownership

2.2.5 Changelog

- Changes in **Version 2**:

- **Fee on Rapid Liquidity Additions:**

A fee is now charged when liquidity is added repeatedly within a short time frame, discouraging spamming that could disrupt price scale adjustments.

- **Donation Share Usage Limitation:**

The number of donation shares available for rebalancing is reduced if liquidity was recently added, mitigating the extraction of value from the donation buffer.

- **Donation Share Burn on Price Scale Adjustment:**

During price scale adjustments, donation shares may be burned until the new virtual price recovers to its previous value, rather than stopping at the threshold virtual price. This ensures that the donation shares are first used, before using the profits from the pool, to adjust the price scale.

- **Improved Fee Calculation:**

Fee calculations are now based on the actual (spot) balances of the pool, rather than the price scale, in line with `remove_liquidity()`. The calculation also incorporates the new spamming protection fee for donations.

- **Donation Withdrawal and Pool Reset Function:**

A new function allows for the withdrawal of donations and resets the pool if only donation shares remain, ensuring the pool can recover from re-initialization.

- Changes in **Version 4**:

- **Vyper Version Upgrade:**

- Bumped the Vyper version to 0.4.3 (and implicitly the EVM version to prague).

- **View Contract Architecture Update:**

- The View contract provided by the factory is no longer used by the pool. Instead, the pool holds the address of the view contract itself, which can be updated by the factory admin with `set_views()`.

- **Enhanced Donation Share Burn Strategy:**

- Donation shares are now burned to recover to the old virtual price, rather than the threshold virtual price, when the price scale is adjusted. This ensures that donation shares are used first to adjust the price scale before using profits from the pool.

- Changes in **Version 5**:

- **Donation cap:**

- Donations are capped at a given percentage of the pool's total supply, preventing excessive donations that could lead to manipulation or exploitation of the pool's mechanics. The parameter controlling the percentage can be updated by the factory admin using `set_donation_protection_params()`.

- Changes in **Version 6**:

- **Math Contract Architecture Update:**

- The math contract address is now stored in the pool contract, allowing the factory admin to update it together with the view contract using `set_periphery()`. This means that the `_math` argument provided in the constructor of the pool is no longer used.

- Changes in **Version 7**:

- **tweak_price frequency limitation:**

- The frequency of the `tweak_price` function calls is now limited to once per block, during the first call to the pool in a given block.

- **Donation share burn strategy adjustment:**

- `tweak_price` has been updated to allow for more frequent burns. Before, once shares were burned, the remaining shares would be made available linearly over time over the donation duration, even if they were previously unlocked. Now, once shares are burned, the remaining shares remain available for burning.

- Changes in **Version 8**:

- **Donation protection parameter tuning:**

- The default donation protection period was reduced from 600 seconds to 60 seconds and the LP threshold was increased from 20% to 50%. The LP spam penalty was also adjusted so that it scales with the current donation share ratio, instead of always applying the full protection-based penalty whenever donation protection is active.

- **Donation protection arithmetic hardening:**

- Several donation protection computations were changed to use explicit `unsafe_div` and `unsafe_sub` operations after the relevant non-zero and bounds checks. The donation protection parameter setter documents that the non-zero checks are required for these unsafe operations.

- Changes in **Version 9**:

- **External Policy contract support:**

Pools can now be configured with an optional Policy contract. The policy can override the dynamic fee returned by `_fee()` and provide a price-scale target for rebalancing. The pool pushes the current state to the policy after relevant operations through `update_pool_state()`.

- `get_fee`: Returns the dynamic fee for a given `xp`, replacing the pool's internal `_fee()`. The default implementation reproduces the `mid_fee/out_fee/fee_gamma` formula by reading `packed_fee_params()` back from the pool.
- `get_price_scale`: Returns the price-scale target `tweak_price` moves toward. The default implementation extrapolates the pool's EMA from the last pushed state and the pool's `ma_time`.
- `update_pool_state`: Pool-only entry point invoked from the pool after fee-affecting operations. Stores `xp`, `price_scale`, `price_oracle`, `last_prices`, `virtual_price`, `xcp_profit`, `D`, and `block.timestamp` into `last_pool_state`.
- The POOL address is immutable and bound at policy deployment, so each policy instance serves a single pool.

- **Rebalancing parameter update:**

The packed rebalancing parameters now represent `adjustment_step_min`, `adjustment_step_max`, and `ma_time`. The former `allowed_extra_profit` threshold is no longer used directly; price-scale adjustments are instead gated by the LP-protected profit threshold and the min/max adjustment step range.

- **Fee and profit accounting rework:**

The fee split is now controlled by `reserved_profit_fraction` and `admin_fee`. Admin fees are accounted in token-denominated `admin_balances` and later transferred to the fee receiver, while `lp_xcp_profit` tracks the LP-protected profit threshold used during rebalancing.

- **Pool initialization and LP allowlist:**

Pools can be deployed in an initialization-required mode. Before the first liquidity deposit, `initialize()` can set the fee split, policy, initial price, and optional LP allowlist. When the allowlist is enabled, non-donation liquidity additions require the sender to be allowlisted.

- **Periphery address handling:**

The math and view contract addresses are immutable in the pool and are expected to be source-patched into the blueprint at deployment time. Existing pools no longer expose a `set_periphery()` function.

- **Donation protection refinement:**

The donation protection defaults were set to a 600 second period and a 20% LP threshold. A remainder accumulator was added so small liquidity additions can still contribute proportionally to future protection extensions.

- Changes in **Version 10**:

- **Policy target bounding:**

Policy-provided price-scale targets are clamped to a 20% band around the pool oracle before the native step limiter is applied. A policy target equal to the current price scale can still hold rebalancing while the price scale remains inside this oracle band, but cannot indefinitely pin the pool outside the band.

- **LP profit threshold update:**

When virtual price decreases, `lp_xcp_profit` is now reduced by the virtual-price delta down to the `1e18` floor instead of being scaled proportionally with `xcp_profit`.

2.3 Trust Model

The following trust model applies to the system as of the latest version of this report.

General:

- Contracts are expected to be deployed on chains supporting the `cancun` or `prague` hardforks.
- The view contract was brought closer to the pool's fee calculation path in [Version 9](#), but integrations should still treat view outputs as previews and account for the integration considerations below, especially around low-liquidity or manipulated pools.
- In the current version of the system, the view and math contract addresses are immutable pool variables that are expected to be source-patched into the blueprint before deployment. If they are left as empty (`address`), the pool is unusable.

Permissionless Deployment:

Anyone can deploy pools via the factory. Users should verify that pools are factory-deployed with sensible parameters, non-malicious tokens, and, if configured, a trusted policy contract. For initialization-required pools, the deployer or factory admin can set the initial fee split, policy, price, and LP allowlist before the first liquidity deposit. The deployer is trusted to set these parameters sensibly, but loses all privileges after initialization.

Integration:

As described in [Integration Considerations](#), integrators should be very careful when integrating with the system, as view functions are previews and can be affected by manipulated or stale pool state. Empty or low liquidity pools are easy to manipulate and can return inconsistent values; integrators might want to consider ensuring that pools cannot be re-initialized by sending LP tokens to a dead address.

Upgradeability Constraints:

While the factory can be given new implementations to be used for future deployments, once deployed, the following upgradeability constraints apply for a given pool:

- **Pool contracts:** Not upgradeable
- **View implementation:** Not upgradeable for existing pools; since [Version 9](#), pools use immutable source-patched view addresses and no longer expose a pool-level periphery setter
- **Math implementation:** Not upgradeable for existing pools; since [Version 9](#), pools use immutable source-patched math addresses and no longer expose a pool-level periphery setter
- **Policy contract:** The admin can call `set_policy_contract` for a Pool.

Supported Tokens:

- The system is intended for ERC-compliant tokens with 18 or fewer decimals.
- Tokens should not exhibit malicious or non-standard behaviors (e.g., rebasing, fee-on-transfer, non-compliant interfaces, hooks, or double entry points).

2.3.1 Roles and Privileges

Factory Admin (fully trusted):

Can update existing pool parameters, fee parameters, policy contract, LP allowlist, fee receiver, factory implementation references, and admin address. In the worst case, the admin could:

- Change an existing pool's parameters to extract value from it or DoS it:
 - Set a malicious or reverting policy contract
 - Set fees up to the protocol maximum
 - Enable or modify the LP allowlist
 - Update other pool parameters to enable value extraction, subject to the configured ramping and parameter bounds, or to DoS a pool.
 - Change the exponential moving average time to a very small value, enabling price oracle manipulations
- Change factory parameters:
 - Pause new deployments
 - Manipulate pool or gauge implementations to create malicious new pools or gauges
 - Change factory-level math and view implementation references used by deployment or integration tooling, noting that **(Version 9)** and later pools use source-patched immutable periphery addresses

Fee Receiver (fully trusted):

Receives collected fees. Has no control over the system but is expected not to misappropriate fees.

Policy Contract (trusted if configured):

Can influence pool fees and the target price scale used during rebalancing.

- If the policy returns extreme values, it can make interactions more expensive, redirect rebalancing behavior, hold the price scale, or prevent rebalancing. As described in the [Policy target prices can pass LP protection while mispricing the pool](#) issue, the policy is trusted to return values within reasonable bounds to avoid these issues.
- If policy fee, target, or required state-update calls revert, swaps, liquidity additions, unbalanced withdrawals, and policy installation on non-empty pools can revert.
- Balanced withdrawals use a best-effort state update and should remain available even if the policy callback fails.
- By returning a wrong target price scale within the oracle-bounded target band, the policy can cause suboptimal price scale adjustments that could be exploited by arbitrageurs and lead to losses for LPs.

End Users (untrusted):

Can interact with pools, donate, and deploy new pools via the factory. They have no special privileges but may strategically sequence swaps, liquidity operations, donations, and deployments. Integrators and users should ensure that pools are deployed with sensible parameters, non-malicious tokens, and, if configured, a trusted policy contract.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation
- **Trust**: Violations to the least privilege principle

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	5

- [Policy Target Prices Can Pass LP Protection While Mispricing the Pool](#) **Code Partially Corrected**
- [Admin Is Able to DoS the System](#) **Code Partially Corrected**
- [Griefing Attack on Liquidity Additions](#) **Risk Accepted**
- [Donations Can Be Extracted With Swaps](#) **Risk Accepted**
- [Parameters Check Mismatch](#) **Risk Accepted**

5.1 Policy Target Prices Can Pass LP Protection While Mispricing the Pool

Trust **Low** **Version 9** **Code Partially Corrected**

CS-CURVE-FXSWAP-V3-051

Pools can be configured with an external Policy contract. During `tweak_price()`, a non-zero value returned by `Policy.get_price_scale()` is used as the rebalancing target instead of the pool's native EMA oracle target:

```
if policy != empty(Policy):
    p_policy = staticcall policy.get_price_scale()

target_price: uint256 = price_oracle
if p_policy > 0:
    target_price = p_policy
```

The selected target is step-limited by `adjustment_step_max`, but the target itself is not bounded against the native oracle. The subsequent LP protection is also evaluated after repricing `xp` with the selected `p_new`:

```
xp: uint256[N_COINS] = [
    _xp[0],
    unsafe_div(_xp[1] * p_new, price_scale)
```

```

]
new_D: uint256 = staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0)
new_xcp: uint256 = self._xcp(new_D, p_new)
new_virtual_price: uint256 = 10**18 * new_xcp // total_supply

if (
    new_virtual_price > 10**18 and
    new_virtual_price >= lp_xcp_profit
):
    self.D = new_D
    self.virtual_price = new_virtual_price
    self.cached_price_scale = p_new

```

This check preserves the watermark in the accounting units induced by the new price_scale. It is therefore not a hard barrier against economically wrong policy targets.

A malicious or compromised policy can exploit this by returning a target far from the real market price. For example, after a trade makes the pool coin0-heavy, the policy can direct price_scale downward. The native step limit caps the movement, but the pool still commits a partial move away from the true 1:1 market if the internal virtual-price check passes.

The check can pass because lowering p_new in that coin0-heavy state makes the repriced xp appear more balanced. new_D can grow enough relative to sqrt(p_new) that new_xcp and new_virtual_price increase, even though the pool has moved toward an economically wrong price. The rebalance is then committed, and because the same call can increase lp_xcp_profit, the watermark ratchets upward under the bad scale.

Once the bad price_scale is committed, arbitrageurs can trade against the mispriced pool at the external market price. In the downward-manipulation case, coin1 is underpriced by the pool, so arbitrageurs sell coin0 and receive excess coin1. The pool gains the overvalued side and loses more of the undervalued side, leaving LPs with a smaller total position when measured at the true market price. The loss can exceed the fee income from the trigger trade because the accepted rebalance itself creates the stale/malicious price surface that arbitrageurs unwind.

This is primarily a policy trust-model issue: the configured policy must be trusted not only for fee behavior and liveness, but also for the economic correctness of every positive price-scale target it returns. The pool-local lp_xcp_profit guard and step limiter do not by themselves ensure that a policy-directed rebalance tracks the native oracle or preserves LP value at the external market price.

Code partially corrected:

As of Version 10, the returned target price is bounded by the native oracle price:

```

if p_policy > 0:
    policy_bound: uint256 = unsafe_div(price_oracle, 5)
    target_price = min(
        max(p_policy, unsafe_sub(price_oracle, policy_bound)),
        price_oracle + policy_bound,
    )

```

This limits the potential damage that can be done by a malicious or compromised policy in a single rebalance, but it does not eliminate the trust requirement on the policy to return economically reasonable targets within that range.

Curve accepted the risk of a policy mispricing the pool within the bounds of the native oracle price.

5.2 Admin Is Able to DoS the System

Trust

Low

Version 2

Code Partially Corrected

CS-CURVE-FXSWAP-V3-030

The admin can accidentally (or intentionally) cause a denial of service (DoS) in the system by setting certain parameters to values that cause arithmetic overflows or reverts in user-facing operations. The following attack vectors have been identified:

Attack Vectors:

1. Protection factor overflow in `_donation_shares`:

The `protection_factor` could exceed `PRECISION`, causing the function to revert. This occurs if `self.donation_protection_period` is set smaller than `(expiry - block.timestamp) * PRECISION`.

A malicious admin could exploit this by:

- First setting a very large `donation_protection_period` and adding liquidity to increase `expiry` to a high value
- Then setting a very small `donation_protection_period`, causing the subtraction to underflow for an extended period, as `expiry` would be far in the future

2. Multiplication overflow in `add_liquidity`:

In `add_liquidity`, the following is computed:

```
protection_period: uint256 = self.donation_protection_period
extension_seconds: uint256 = min(relative_lp_add * protection_period // self.donation_protection_lp_threshold, protection_period)
```

By setting a very large `donation_protection_period`, the admin could make `relative_lp_add * protection_period` consistently revert due to multiplication overflow, preventing any liquidity from being added to the pool.

3. Multiplication overflow in `tweak_price`:

In the latest version, the admin can configure `donation_duration` via `set_donation_parameters()`. The function only checks that the duration is non-zero and does not enforce any maximum bound:

```
assert duration > 0 # dev: "donation duration cannot be zero"
self.donation_duration = duration
```

When a price-scale adjustment burns donation shares, `tweak_price()` recomputes the donation release timestamp with the following expression:

```
if new_total > 0 and shares_unlocked_new > 0:
    new_elapsed = (shares_unlocked_new * self.donation_duration) // new_total
```

By setting `donation_duration` to an arbitrarily large value while donations are present, the admin can make the multiplication `shares_unlocked_new * self.donation_duration` overflow whenever the rebalancing path burns donation shares and leaves a non-zero `shares_unlocked_new`. This reverts `tweak_price()` and can DoS swaps and non-balanced liquidity operations that attempt such a rebalance.

4. Reverting policy contract:

In the latest version, the admin or deployer can configure a policy contract. If the configured policy reverts in `get_fee()`, `get_price_scale()`, or `update_pool_state()`, most user-facing operations can be blocked.

In particular, swaps, liquidity additions, and non-balanced liquidity removals revert when the policy callback reverts. By contrast, balanced liquidity removals do not revert on policy callback failure, and should always succeed regardless of the policy's behavior.

Code partially corrected:

- **Issue 1, 2 and 3** where fixed by adding appropriate boundary checks.
- **Issue 4** remains present in the latest version as the admin can configure a policy contract whose required calls revert.

5.3 Griefing Attack on Liquidity Additions

Design **Low** **Version 2** **Risk Accepted**

CS-CURVE-FXSWAP-V3-032

The new spamming protection increases the fee for repeated liquidity additions in a short time frame. Hence, a malicious actor could front-run a liquidity addition to increase the fee for the next liquidity addition. Depending on the addition size and the initial size of the pool, this could be profitable for the attacker, as the fee is partially redistributed to the liquidity providers.

Currently, the `donation_protection_expiry_ts` is set even when there is no donation ongoing. Still, subsequent liquidity additions would be charged with a spamming protection fee.

Risk accepted:

Curve changed the code such that the `donation_protection_expiry_ts` is not set when there is no donation. The remaining risk of a griefing attack is accepted by Curve.

5.4 Donations Can Be Extracted With Swaps

Design **Low** **Version 1** **Risk Accepted**

CS-CURVE-FXSWAP-V3-028

Donations are designed to slowly drip over time. This means that over time, rebalancing operations that were not possible before due to the donation buffer being too low can be executed once the donation buffer has grown large enough. If the rebalancing conditions are not met for a long time, the donation buffer will still grow to the point of being fully available.

In such a case, for the next price tweak, the donation buffer could be used to compensate a relatively high price adjustment. These Curve changes can be exploited to extract donations from the pool by a trader by trading out and away from the equilibrium on the old Curve prior to the rebalancing and back in the direction of the equilibrium on the new Curve. The profitability of the attack is dependent on the possible price scale adjustment.

Risk accepted:

Curve has accepted the risk of this issue. They added the following comment:

We altered the donation absorption logic to use up more of donations and potentially limit price jumps, thus reducing the risk.

5.5 Parameters Check Mismatch

Correctness

Low

Version 1

Risk Accepted

CS-CURVE-FXSWAP-V3-008

In both `TwocryptoFactory.deploy_pool()` and `Twocrypto.apply_new_parameters()`, new parameters can be set for a pool. However, the checks for the parameters are not consistent between the two functions:

1. The factory checks that `mid_fee < MAX_FEE-1`, while the pool checks that `mid_fee < MAX_FEE + 1`.
 2. The factory checks that `out_fee < MAX_FEE-1`, while the pool checks that `out_fee < MAX_FEE + 1`.
 3. The factory checks that `fee_gamma < 10**18 + 1`, while the pool checks that `fee_gamma < 10**18`.
 4. In the factory, `adjustment_step` is ensured to be non-zero, while the pool does not check this.
-

Risk accepted:

Client has acknowledged this issue.

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	1
• Donations Can Be Extracted Code Corrected	
Medium -Severity Findings	7
• Policy Callbacks Can Be Skipped by Underfunding the Capped Call Code Corrected • Policy and Pool Timestamp Drift on Policy State Updates Code Corrected • Donations Can Be Directly Absorbed by the Pool Code Corrected • Inconsistent LP Operations and Resulting Fees Code Corrected • Last Part of Ramping Can Be Extracted Code Corrected • Last Removal Can Block the Pool Code Corrected • Ramping Can Break the Pool Code Corrected	
Low -Severity Findings	12
• Ramping Pool Can Become Stuck at the Imbalance Boundary Code Corrected • Mid Fee Can Be Set to Zero Code Corrected • calc_token_amount Applies Withdrawal Fees in the Wrong Direction Code Corrected • _calc_withdraw_one_coin() Does Not Account for NOISE_FEE Code Corrected • Donation Release Can Be Artificially Delayed Code Corrected • Incorrect Approximate Fee Code Corrected • Incorrect Fee Formula Code Corrected • Incorrect MIN_A and MAX_A Values Code Corrected • Last Part of Ramping Can Be Extracted When Stopping Ramping Code Corrected • Re-initialization of a Ramping Pool Code Corrected • View Functions for withdraw_one_coin Are Incorrect Code Corrected • calc_token_fee() Incorrect for Removal of Liquidity Code Corrected	
Informational Findings	14
• Full Admin Fee Settings Can Make the Fresh Virtual Price Check Reject Operations Code Corrected • Ramping Can Be Started Before the Pool Is Initialized Code Corrected • Zero-Amount Withdrawals Skip Policy Update Code Corrected • Unsafe Use of unsafe_div Code Corrected • Protection Active When No Shares Code Corrected • donation_protection_expiry_ts Not Reset Code Corrected	

- Donation Sandwiching Through Pool Re-initialization **Code Corrected**
- Function Mutability Too Broad **Code Corrected**
- Incomplete and Unclear Event **Code Corrected**
- Inconsistent approx_fee Unit **Code Corrected**
- Misleading Variable Name **Code Corrected**
- Re-initialization of Pool Zeroes Out Admin Fees **Code Corrected**
- _calc_dtoken_nofee Does Not Handle First Deposit **Code Corrected**
- wad_exp Implicit Cast **Code Corrected**

6.1 Donations Can Be Extracted

Design **High** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-001

With the new donation system, rebalancing can use donated funds to cover losses produced by the rebalancing operation.

Donation works as follows:

1. The system computes `donation_shares_to_burn` such that `new_vp >= threshold_vp` where:

- `new_vp = 10**18 * new_xcp // (total_supply - donation_shares_to_burn)`
- `threshold_vp = 10**18 + (xcp_profit - 10**18) // 2`

2. The system burns `donation_shares_to_burn` shares from the donation pool:

```
if donation_shares_to_burn > 0:
    # we burned some donation shares, update related state
    self.donation_shares -= donation_shares_to_burn
    self.totalSupply -= donation_shares_to_burn
    self.last_donation_release_timestamp = block.timestamp
```

Exploitation Vector

By burning the donation shares, the price of shares owned by LPs increases, as the fraction of assets to be withdrawn by LPs is computed (directly or indirectly) as (`lp_token_amount / token_supply`).

This mechanism creates an opportunity for attackers to extract donations through the following steps:

1. **Add Liquidity:** Deposit assets to the pool in a balanced way (no fees charged for balanced deposits)
2. **Trigger Rebalancing:** Wait for or trigger the rebalancing operation during the `tweak_price` call
3. **Extract Value:** Withdraw liquidity in the same proportion as deposited, receiving more tokens than originally deposited due to the increased share price from burned donation shares

Impact

This vulnerability allows anyone to extract donations from the pool, potentially leveraging flash loans to amplify the attack without requiring initial capital.

Code corrected:

The issue has been mitigated by adding a mechanism that:

- Retains the donation shares after adding liquidity, to prevent a rebalancing operation from happening immediately after someone adds liquidity. Depending on the amount added, a release timestamp is set, and donation shares will linearly be released over time up to that timestamp. With the current defaults (`donation_protection_period` of 600 seconds and a `donation_protection_lp_threshold` of 20%), the protection reaches its maximum of 10 minutes in the future when adding at least 20% of the total supply, but these parameters can be adjusted.
- Increases fees when adding liquidity after other users have added liquidity, to prevent users from DOS-ing rebalancing by adding and removing liquidity in a balanced way again and again.

Note that the new design does not fully fix the issue, but rather mitigates it to make it less exploitable, as it is no longer possible to perform the attack in a single transaction, and capital has to be locked for a longer period of time that might be extended as new donations are made.

6.2 Policy Callbacks Can Be Skipped by Underfunding the Capped Call

Design Medium Version 9 Code Corrected

CS-CURVE-FXSWAP-V3-046

The balanced `remove_liquidity()` path updates the external Policy state on a best-effort basis. It calls `_update_policy_state(..., must_succeed=False)` after burning LP tokens, reducing D, and transferring the withdrawn balances:

```
self._update_policy_state(
    self.xp(self.balances, price_scale),
    price_scale,
    self.cached_price_oracle,
    self.last_prices,
    self.virtual_price,
    self.xcp_profit,
    self.D,
    False,
)
```

In this mode, `_update_policy_state()` forwards at most 250_000 gas to the policy and ignores the returned success flag:

```
success: bool = raw_call(
    policy.address,
    data,
    max_outsize=0,
    gas=250_000,
    revert_on_failure=False,
)
```

The 250_000 value is only an upper bound. Because external calls are still subject to EIP-150 gas forwarding, the policy receives `min(gasleft() * 63 / 64, 250_000)` rather than an unconditional 250_000. An account can therefore submit a balanced withdrawal with just enough gas for the pool to finish after the callback returns, while leaving less gas than the policy needs to complete

`update_pool_state()`. In that case, the policy call fails out of gas, success is ignored, and the withdrawal still succeeds.

This allows a caller to deliberately avoid synchronizing the policy after a balanced liquidity removal. For example, if the policy callback needs K gas, a transaction can be sized so the callback receives $K - 1$ gas, fails, and returns control with enough remaining gas for the pool to emit `RemoveLiquidity` and return. The policy then keeps a stale snapshot of `xp`, `D`, `virtual_price`, and price-oracle inputs even though the pool state has changed.

Impact:

The pool treats balanced removals as safe withdrawals that must not be blocked by a broken policy, but the current gas handling also lets arbitrary callers force otherwise valid policy updates to fail. Subsequent policy fee or price-scale decisions can be based on stale pre-withdrawal state until another operation successfully refreshes the policy snapshot.

Code corrected:

The `must_succeed=False` branch of `_update_policy_state()` was changed to:

```
gas_before_policy_call: uint256 = msg.gas
success: bool = raw_call(
    policy.address,
    data,
    max_outsize=0,
    gas=300_000,
    revert_on_failure=False,
)
if not success:
    # Best-effort updates may fail, but the caller must provide
    # enough gas that policy OOG cannot be forced while leaving
    # enough gas for the pool to finish the balanced withdrawal.
    assert gas_before_policy_call >= 500_000
```

This provides two protections:

- Against a malicious policy that could try to force OOG by consuming all the gas provided to it to DoS users who want to perform a balanced withdrawal: the `gas=300_000` parameter ensures the policy has a fixed upper bound on the gas it can consume, so there exists a gas limit that allows the balanced withdrawal to succeed even if the policy call fails.
- Against a malicious user who could try to force OOG by providing just enough gas for the balanced withdrawal to succeed while the policy call fails: the assertion after the call ensures that if the policy call fails, the user provided at least 500_000 gas. Given 300_000 as an upper bound on the gas required for an honest policy to always succeed, the pool should have $300_000 * 64/63$ gas left right before the call to the policy, after accounting for operations such as the `CALL` opcode cost itself and the ABI encoding of the arguments. For that reason, 500_000 provides a large margin to account for any such operations.

6.3 Policy and Pool Timestamp Drift on Policy State Updates

Design Medium Version 9 Code Corrected

CS-CURVE-FXSWAP-V3-047



The Twocrypto pool calls `_update_policy_state()` on most state-changing entry points to push fresh state to the external Policy contract. The policy stores its own `state.ts`, while the pool maintains `self.last_timestamp` as the EMA clock used by `tweak_price()` and `_price_oracle()`. Multiple paths advance `policy.state.ts` without advancing `self.last_timestamp`, allowing the two clocks to drift apart.

Path 1 — Balanced remove_liquidity:

```
# remove_liquidity (balanced removal) -- Twocrypto.vy
self._update_policy_state(
    ...
    self.virtual_price,
    ...
    self.D,
    ...
    must_succeed=False,
)
```

Balanced `remove_liquidity` deliberately does not call `tweak_price()`, so `self.last_timestamp` is not touched. `_update_policy_state`, however, sets `policy.state.ts = block.timestamp` on every successful callback.

Path 2 — First deposit:

```
# add_liquidity, first-deposit branch (old_D == 0) -- Twocrypto.vy
self.D = D
self.virtual_price = 10**18
...
self.mint(self, MINIMUM_LIQUIDITY)
self.mint(receiver, d_token)
self._update_policy_state(...)
```

The first-deposit branch never assigns `self.last_timestamp`; the slot keeps the value written by the constructor. `_update_policy_state` again advances `policy.state.ts` to the current block.

Path 3 — Setting a fresh policy on a non-empty pool:

```
# set_policy_contract -- Twocrypto.vy
self._set_policy(policy)
if policy != empty(Policy) and self.D > 0:
    self._update_policy_state(
        self._xp(self.balances, self.cached_price_scale),
        self.cached_price_scale,
        self.cached_price_oracle,
        self.last_prices,
        self.virtual_price,
        self.xcp_profit,
        self.D,
        True,
    )
```

When the admin installs a fresh policy on a non-empty pool, the pool pushes the current cached state to the policy and the policy stamps it with `block.timestamp`. This path does not call `tweak_price()` and therefore does not refresh `self.cached_price_oracle` or advance `self.last_timestamp`. If the pool's timestamp is stale, the newly installed policy starts from stale oracle inputs but with a fresh policy timestamp.

Implications:



After any of these paths, `policy.state.ts == block.timestamp` while `self.last_timestamp` still holds an older value. The next call to `tweak_price()` (e.g. on the following swap) will compute an EMA alpha from `block.timestamp - self.last_timestamp`, treating the gap as if no state update had occurred for the policy either. The policy, in contrast, will return a `get_price_scale()` based on its own freshly-stamped state. The two views of "elapsed time since last sample" disagree.

Code corrected:

`TwocryptoPolicy.update_pool_state` now accepts an additional `oracle_timestamp` argument, and the pool passes `self.last_timestamp` for that parameter on all calls.

6.4 Donations Can Be Directly Absorbed by the Pool

Design Medium Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-002

In the function `add_liquidity`, donations are handled with the following logic:

```
if is_donation:
    if self.donation_shares == 0:
        # first donation should initialize last_donation_release_timestamp
        self.last_donation_release_timestamp = block.timestamp

    # if we donate, we don't explicitly mint lp tokens, but we add to the donation shares and total supply
    self.donation_shares += d_token
    self.totalSupply += d_token
```

However, this means that if the pool already has donations that were not yet consumed, the new donation will not be absorbed by the pool over time, but will instead be directly unlocked and available for rebalancing as `last_donation_release_timestamp` is not updated.

Code corrected:

As of [Version 2](#), the code was refactored. New donations will be combined with old donations such that the drop-in (tokens per second released) would be the same as if the combined amount had been donated from the beginning.

`_donation_shares()` will release shares following a slope (ignoring the protection feature) defined by:

- `last_donation_release_ts`: The timestamp of the last donation, where the slope crosses the x-axis.
- `donation_duration`: The duration over which the donations are released.
- `donation_shares`: The total shares that should be released at timestamp `last_donation_release_ts + donation_duration`.

A new donation will change the slope in the following way:

- `donation_shares` is increased by the newly added shares.
- `last_donation_release_ts` is set so that at the time of the new donation, `_donation_shares()` remains constant, despite the updated `donation_shares`.

Given the formula in `add_liquidity()`:

```
new_elapsed: uint256 = self._donation_shares(False) * self.donation_duration // new_donation_shares
```

This means that under regular donation patterns (e.g., regular donations of constant amounts), and assuming no rebalancing happening, the `_donation_shares()` function converges to `donation_shares`, meaning virtually all donated tokens become available over time.

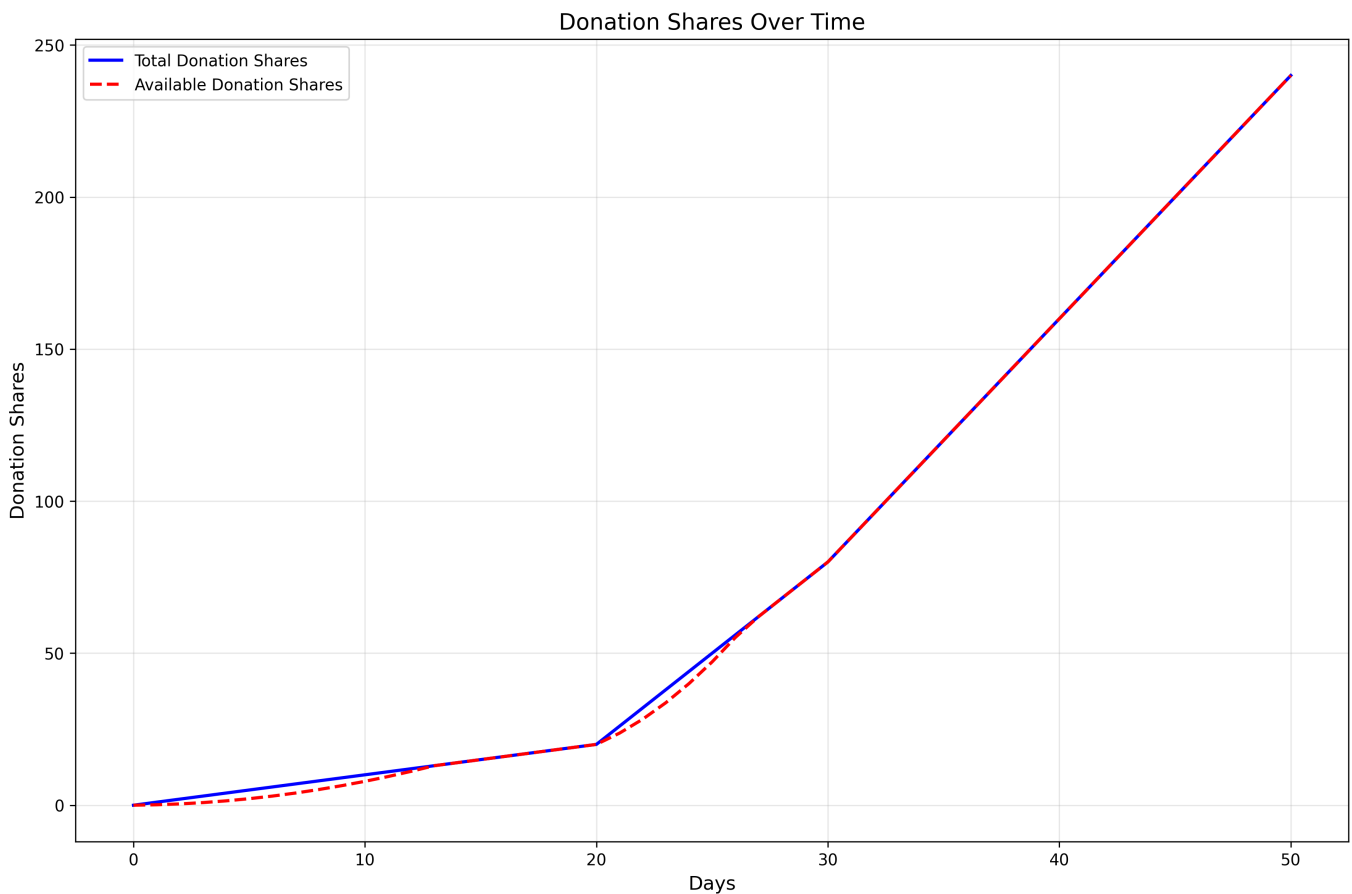
The virtual donation mechanism creates a feedback loop where:

1. Each new donation resets the vesting timeline to preserve currently unlocked amounts.
2. The reset magnitude decreases as total donations grow:

$$\text{new_elapsed} = \text{unlocked_shares} * \text{duration} / \text{total_donations}.$$
3. As the donation pool grows, resets become proportionally smaller while time continues advancing.

This means that the system converges towards a steady state over time where the donation unlock rate matches the donation input rate.

Below is a graph comparing `_donation_shares()` with `donation_shares` over time. One donation happens each day, for the first 20 days: 1 unit/day, for the next 10 days: 6 units/day and for the last 20 days: 8 units/day.



This behavior is expected and desired by Curve, which states:

Donations absorbed directly only if a significantly larger amount is already available. This is by design. Adding 1 share when already 100 are unlocked should not drip this one share over next 7 days.

6.5 Inconsistent LP Operations and Resulting Fees

Correctness

Medium

Version 1

Code Corrected

CS-CURVE-FXSWAP-V3-003



The different operations and their definition of "balanced":

When adding liquidity, the user's contribution to the pool is defined as the delta in D from the previous state. In case a user adds liquidity in a balanced way, no or the lowest possible fees (`NOISE_FEE`) should be applied. In `add_liquidity` this is the case when `token_amounts` are added in a balanced way with respect to the current `price_scale`. This contribution would give the lowest possible fees. The user will receive their D contribution in LP tokens.

Liquidity can be removed with either:

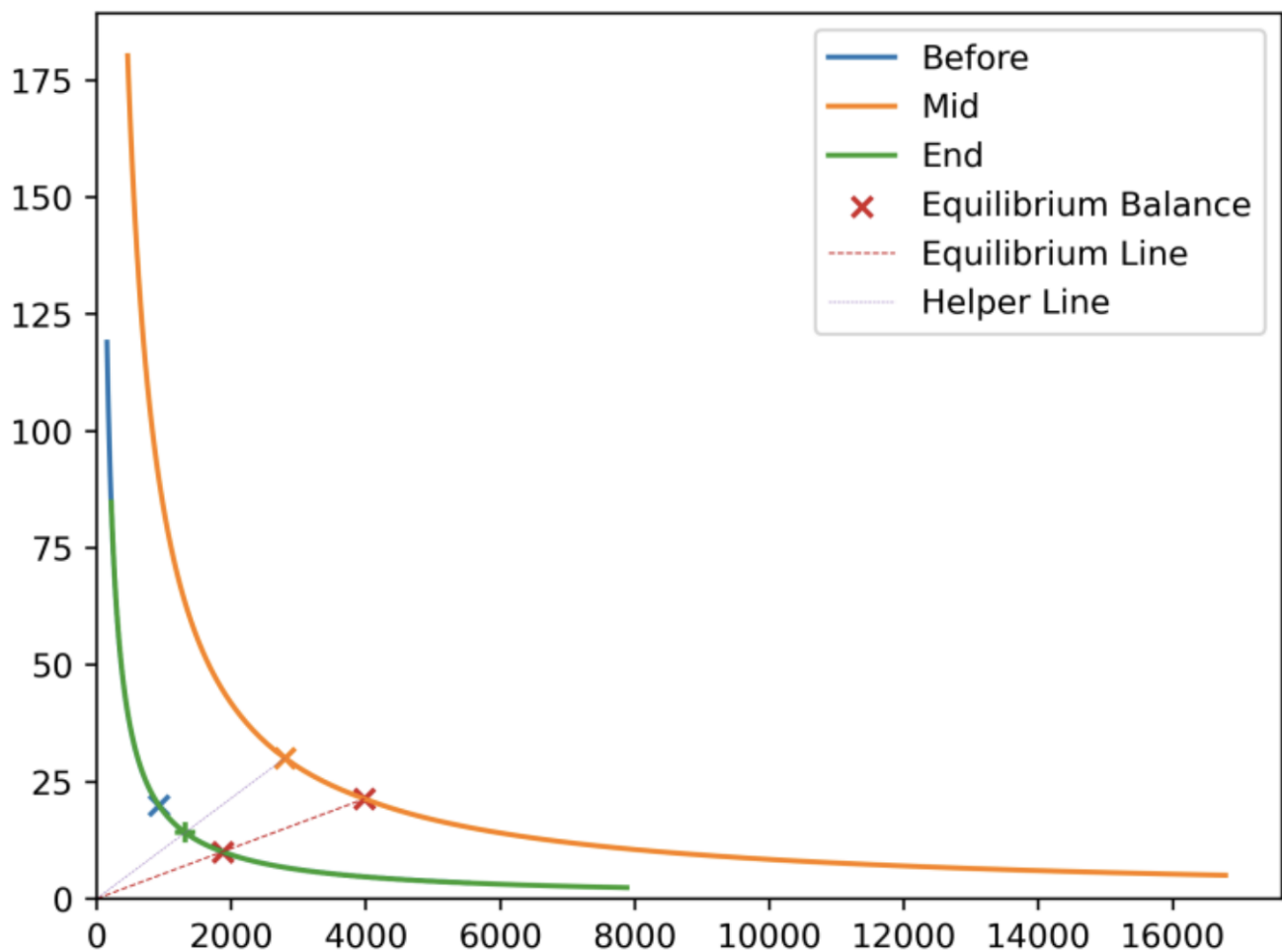
- `remove_liquidity` which should be the safe way to withdraw liquidity. To avoid any issues, no fees are charged as it removes liquidity proportionally. Proportional in this case is defined to be a withdrawal with a scaling factor f applied to the current amount of tokens in the pool, such that the new token amounts in the pool transition from x_0, x_1 to $f * x_0, f * x_1$. This is due to the fact that a user's LP share is simply multiplied by the token amounts.
- `remove_liquidity_fixed_out` allows the user to withdraw funds in a defined ratio and charges fees on the withdrawal. But a user could avoid fees by withdrawing with equal values for both tokens. This implicitly defines a balanced withdrawal with respect to the current `price_scale` (like in `add_liquidity`).

In summary, there are two different definitions of balanced liquidity operations. One is defined by the current `price_scale` and the other is defined by the current token amounts (x).

Implications:

Due to this inconsistent behavior, adding liquidity will make an implicit trade in most scenarios and, hence, will also trigger a price change. An attacker could exploit this behavior to trade without paying fees as described in our CurveCryptoSwap2ETH report from April 1st 2022:

If a pool is currently not in the equilibrium state (as defined by `price_scale`), then the trade fees can be avoided when trading towards the equilibrium. This is possible as follows:



We assume that the blue X represents the initial pool state. The attacker can move the pool to the green cross, and hence perform a trade, without paying the trading fees. The steps are:

1. The attacker adds liquidity in the balance defined by the `price_scale`. Hence, the pool state moves to the orange X. During this step only the much smaller `NOISE_FEE` is paid as a fee, as it is a balanced addition.
2. The attacker removes liquidity using `remove_liquidity`. Hence, no fee is paid. However, the liquidity is removed according to the current pool balances which are different from the ratio of `price_scale`. Thus, the resulting state is the green cross.

Hence, overall a trade occurred, but the attacker only paid the small `NOISE_FEE` and not the trade fee. As a consequence, the pool misses out on those fees. However, this only works if the attacker has access to the tokens needed for the additional liquidity.

Other implications include:

- Adding and removing liquidity with `remove_liquidity` in a balanced way is not a no-op and will not result in the initial state (Not accounting for the `NOISE_FEE`).
- Adding and removing liquidity with `remove_liquidity_fixed_out` will result in the initial state if the removal is done in a balanced way with respect to the current `price_scale` (Not accounting for the `NOISE_FEE`).

Code corrected:

The code has been corrected to ensure that the definition of "balanced" is consistent across all liquidity operations. The raw balances are now used as the meaning of "balanced" for all operations. Note that:

- For `add_liquidity`, the raw balances before the operation are used to determine if the addition is balanced.
- For `_calc_withdraw_fixed_out`, the raw balances after the operation are used to determine if the removal is balanced.

Additionally, a fee is charged for each operation depending on how the pool is balanced regarding the price scale (`_fee()` function). This is designed to have the pool make more profits in such case, helping with the rebalancing of the pool.

6.6 Last Part of Ramping Can Be Extracted

Correctness **Medium** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-036

When ramping, the first step of each function is to recompute D based on the new A and gamma values. This is done before any other computation, and before the user's input is processed, for example, in `_exchange()`:

```
if self._is_ramping():
    x0 *= PRECISIONS[i]

    if i > 0:
        x0 = unsafe_div(x0 * price_scale, PRECISION)

    x1: uint256 = xp[i] # <----- Back up old value in xp ...
    xp[i] = x0 # |
    self.D = staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0) # |
    xp[i] = x1 # <----- ... and restore.
```

Where the `_is_ramping()` function is defined as follows:

```
@internal
@view
def _is_ramping() -> bool:
    """
    @notice Checks if A and gamma are ramping.
    @return bool True if A and/or gamma are ramping, False otherwise.
    """
    return self.future_A_gamma_time > block.timestamp
```

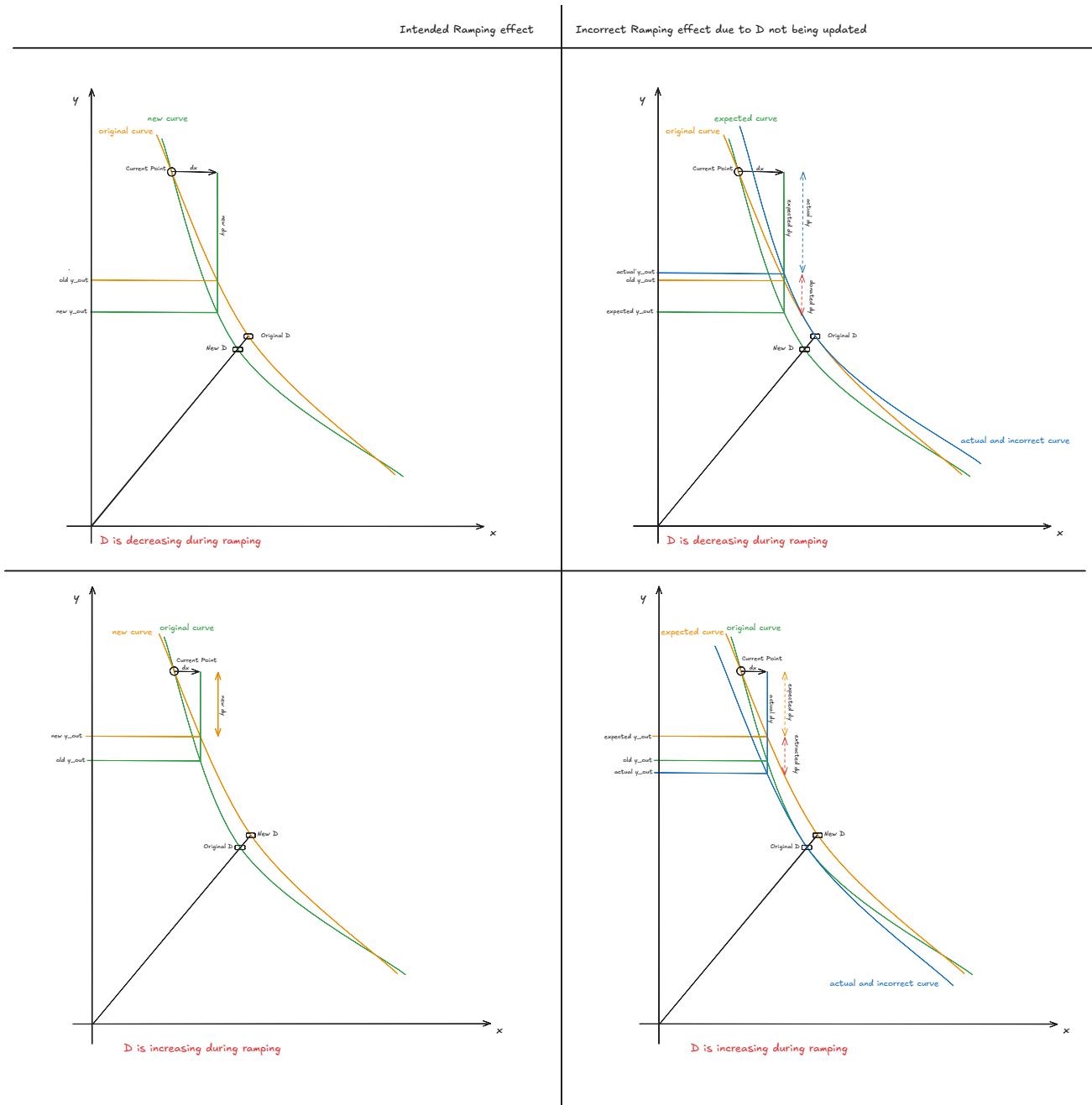
Issue:

The `_is_ramping()` check will return `False` if the ramping period has ended. However, this becomes an issue when the latest A and gamma values have not yet been applied, specifically, when it is the first call after the ramping period has finished. Consider the following timeline:

- **t0**: Ramping starts, `future_A_gamma_time` is set to `t0 + ramping_duration`.
- **t1**: User calls `exchange()`, which checks `_is_ramping()` and finds it to be `True`, hence it computes D based on new A and gamma values prior to the user's action.
- **t2**: Ramping period ends (`t2 = t0 + ramping_duration`).
- **t3**: User calls `exchange()`, which checks `_is_ramping()` and finds it to be `False`, hence it does not recompute D and uses the old D value stored at the end of the previous call (at t1).

This means that after the ramping period ends, the first call to the pool will perform an action on the old and invalid D value, which was obtained during the last call within the ramping period.

The following diagram illustrates the issue:



• On the left side, the two charts show what should happen in a correct ramping scenario:

- **Top chart:** When the ramping leads to a decrease of D
- **Bottom chart:** When the ramping leads to an increase of D

• On the right side, the two charts show what happens in the current implementation:

- **Top chart:** When the ramping leads to a decrease of D
- **Bottom chart:** When the ramping leads to an increase of D

In each of the charts:

- The **orange curve** represents the curve obtained after the user action at time t_1 .

- The **green curve** represents what would be the expected curve obtained in the call at time t_3 , and before the user action.
- The **blue curve** represents the actual curve obtained in the call at time t_3 , and before the user action with the current implementation.

Observations:

- **For the top charts (decrease of D):** The first user calling after the ramping period is using a curve with a higher D value than expected, which means that the user is getting fewer tokens than expected when trading.
- **For the bottom charts (increase of D):** The first user calling after the ramping period is using a curve with a lower D value than expected, which means that the user is getting more tokens than expected when trading, effectively extracting value from the pool.

Similarly, when using `add_liquidity()`, `remove_liquidity()`, or `_remove_liquidity_fixed_out()`, the first user calling after the ramping period is using an outdated curve with a different D value than expected, which can lead to the user getting more or fewer LP tokens or underlying tokens than expected.

Impact:

Depending on the direction of the ramping, the first user calling after the ramping period can either:

- **Extract value from the pool:** When ramping leads to an increase in D, users benefit from trading against an outdated curve with lower D values
- **Lose value:** When ramping leads to a decrease in D, users suffer from trading against an outdated curve with higher D values

The severity of this impact is inversely correlated with pool activity levels:

- **Pools with higher activity** are less affected, as the interval between the last call during ramping and the first call after ramping is short. As a result, the difference between the outdated D and the expected recomputed D is minimal, since the change in A and gamma is small.
- **Pools with lower activity** are more severely impacted. In the extreme case where no calls occur during the ramping period, the parameter update may be entirely missed: the first call after ramping uses an outdated D value, computed before ramping even began.

Code corrected:

`_is_ramping()` now relies on `self.last_timestamp` instead of `block.timestamp` to determine if the ramping period has ended. This ensures that the latest A and gamma values are applied correctly before any user action is processed.

6.7 Last Removal Can Block the Pool

Correctness

Medium

Version 1

Code Corrected

CS-CURVE-FXSWAP-V3-004

The function `remove_liquidity` defines a special case for the last liquidity provider that removes all shares from the pool:

```
if amount == total_supply: # <----- Case 2.
    for i: uint256 in range(N_COINS):

        withdraw_amounts[i] = self.balances[i]
```

However, this case does not account for the `donation_shares`. As a result of this, if the pool still has donations that were not consumed, the last liquidity provider will not enter this case as they will not have the full supply of shares. This will lead to the pool being left in a state where it has no liquidity providers, but still has donations. As such, `D` is not reset to 0.

In that state, any action involving `tweak_price()` (such as `add_liquidity()` or `exchange()`) will fail with a division by zero error as soon as all remaining donations are "absorbed", since `locked_supply` will be 0:

```
donation_shares: uint256 = self._donation_shares()
...
locked_supply: uint256 = total_supply - donation_shares
...
vp_boosted: uint256 = 10**18 * xcp // locked_supply
```

This means that the pool will be effectively blocked.

Code Corrected:

A new sweep function was added to withdraw all remaining tokens from the pool as soon as `self.donation_shares == self.totalSupply`. Hence, the pool is reset with no funds left when the last user withdraws their liquidity. The assets owned by the donation buffer are transferred to the `fee_receiver` address or the factory's admin.

In the latest version of the code, this sweep function was removed. Instead, the pool can no longer be fully emptied because some shares are minted to the pool itself at initialization and are never burned when the last user exits, so a non-zero amount of locked supply always remains.

6.8 Ramping Can Break the Pool

Correctness

Medium

Version 1

Code Corrected

CS-CURVE-FXSWAP-V3-042

When ramping, depending on the direction, the pool might make a profit or a loss. In case it makes a loss, `self.D` decreases to reflect the loss.

```
@internal
@view
def _get_D(A_gamma: uint256[2], xp: uint256[N_COINS]) -> uint256:
    # Normally we need self.D, however, if A and/or gamma are ramping,
    # we need to recalculate D using the current A and gamma values.
    if self._is_ramping():
        # ongoing ramping, recalculate D
        return staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0)
    else:
        # not ramping, use self.D from storage
        return self.D
```

In `tweak_price()`, `xcp_profit` is updated as follows:

```
old_virtual_price: uint256 = self.virtual_price
xcp: uint256 = self._xcp(D, price_scale)

virtual_price: uint256 = 10**18 * xcp // total_supply
```

```

# Virtual price can decrease only if A and gamma are being ramped.
# This does not imply that the virtual price will have increased at the
# end of this function: it can still decrease if the pool rebalances.
if virtual_price < old_virtual_price:
    # If A and gamma are being ramped, we allow the virtual price to decrease,
    # as changing the shape of the bonding curve causes losses in the pool.
    assert self._is_ramping(), "virtual price decreased"

# xcp_profit follows growth of virtual price (and goes down on ramping)
xcp_profit: uint256 = self.xcp_profit + virtual_price - old_virtual_price
self.xcp_profit = xcp_profit

```

Issue:

As highlighted in the comment, during ramping, the virtual price can decrease, leading `xcp_profit` to also decrease. If `xcp_profit` decreases below 10^{18} , which can happen if the ramping occurs in an early stage of the pool, it can lead to breaking the pool due to an underflow in the following line:

```
threshold_vp: uint256 = 10**18 + (xcp_profit - 10**18) // 2
```

Impact:

This issue can effectively break pool functionality when ramping causes losses in early-stage pools, requiring manual intervention to restore normal operation.

Recovery:

To recover from the situation, either:

- The admin should ramp the pool in the opposite direction, which will increase `xcp_profit` and allow the pool to recover.
- A user should perform a very inefficient exchange to increase `xcp_profit` above 10^{18} , which will allow the pool to recover.

Code corrected:

The `threshold_vp` computation was updated from:

```
threshold_vp: uint256 = 10**18 + (xcp_profit - 10**18) // 2
```

to

```
threshold_vp: uint256 = max(10**18, (xcp_profit + 10**18) // 2)
```

Since:

```

10**18 + (xcp_profit - 10**18) / 2
= 2 * 10**18 / 2 + (xcp_profit - 10**18) / 2
= (xcp_profit + 10**18) / 2

```

Note that the computation was later changed to use a different formula.

6.9 Ramping Pool Can Become Stuck at the Imbalance Boundary

Correctness **Low** **Version 9** **Code Corrected**

CS-CURVE-FXSWAP-V3-048

During ramping, most operations recompute D from the current balances before processing the user's action. For example, `_exchange()` does this when `_is_ramping()` returns true:

```
if self._is_ramping():
    ...
    self.D = staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0)
```

The math contract rejects highly imbalanced xp values:

```
assert (
    _xp[0] > 0 and
    _xp[1] > 0 and
    unsafe_div(max(_xp[0], _xp[1]), min(_xp[0], _xp[1])) < 10_000
), "!balance"
```

This creates a liveness edge case while the pool is ramping. A trade can leave the pool very close to the 10_000 imbalance boundary and still pass the post-operation check in `tweak_price()` because the check is performed on the operation's cached xp values. The persisted token balances can then differ slightly from those cached values due to descaling and rounding around the transfer-out path. If the next call reconstructs xp from the stored balances and that reconstructed value is outside the accepted range, the caller can revert during the pre-operation ramp-aware D recomputation, before their action can move the pool back toward balance.

In this state, swaps, liquidity additions, and unbalanced removals can all be blocked while the ramp remains active. Balanced `remove_liquidity()` may still be usable because it intentionally avoids invariant math, but recovery can depend on users withdrawing in a way that happens to bring the remaining balances back under the limit, or on admin intervention.

This is especially relevant for pools used as price-oracle sources: if the state-changing paths that update oracle state are blocked by the imbalance assertion, downstream integrations can observe a stale or unusable pool until the imbalance is resolved.

Code corrected:

The imbalance check at the end of `tweak_price` has been strengthened to:

```
# Pool-level post-operation guard. This is intentionally tighter than the
# 10_000:1 Newton guard in math, which remains the hard numerical limit.
assert (
    xp[0] > 0 and
    xp[1] > 0 and
    unsafe_div(max(xp[0], xp[1]), min(xp[0], xp[1])) < 1_000
), "!balance"
```

This makes the post-check more strict than the pre-check in `math`, creating a buffer that should prevent operations from leaving the pool in a state where the next operation would be blocked by the imbalance check during ramping.

6.10 Mid Fee Can Be Set to Zero

Correctness **Low** **Version 8** **Code Corrected**

CS-CURVE-FXSWAP-V3-049

In `Twocrypto.apply_new_parameters()`, `new_out_fee` is checked against `MIN_FEE` but `new_mid_fee` is only required to be at most `new_out_fee`. Therefore, an admin can set `mid_fee` to zero:

```
assert new_out_fee > MIN_FEE - 1
assert new_mid_fee <= new_out_fee
```

Since `mid_fee` is the fee used near pool equilibrium, balanced swaps can incur zero fees. This is inconsistent with the explicit minimum fee enforced for `out_fee`.

Code corrected:

Both the constructor and `apply_new_parameters()` now require `mid_fee` to be at least `MIN_FEE`.

6.11 `calc_token_amount` Applies Withdrawal Fees in the Wrong Direction

Correctness **Low** **Version 8** **Code Corrected**

CS-CURVE-FXSWAP-V3-050

`Twocrypto.calc_token_amount()` is documented as returning the LP tokens minted for deposits or burned for withdrawals, including fees:

```
@notice Calculate LP tokens minted or to be burned for depositing or
        removing `amounts` of coins
@dev Includes fee.
```

For withdrawals, `TwocryptoView._calc_dtoken_nofee()` first subtracts the requested token amounts from `xp` and returns the no-fee LP burn amount:

```
if deposit:
    d_token -= token_supply
else:
    d_token = token_supply - d_token
```

`calc_token_amount()` then subtracts the fee from that value regardless of the operation direction:

```
d_token -= (
    staticcall Curve(swap).calc_token_fee(amounts, xp, donation, deposit) * d_token // FEE_PRECISION + 1
)
```

This is correct for deposits, where fees reduce the LP tokens minted. It is wrong for withdrawals, where the fee means a user must burn more LP tokens to receive the same target token amounts. Subtracting the fee therefore under-quotes the burn amount returned by `calc_token_amount(amounts, False)`.

On the other side, the main fixed-output withdrawal path applies the fee in the opposite direction. `_calc_withdraw_fixed_out()` starts from the caller-provided LP burn amount, converts it to `dD`, and then reduces the effective invariant decrease by the fee:


```
approx_fee: uint256 = self._calc_token_fee(amounts, xp_new)
dD -= dD * approx_fee // FEE_PRECISION + 1
```

Code corrected:

The fee application in `calc_token_amount()` was changed to:

```
fee: uint256 = staticcall Curve(swap).calc_token_fee(amounts, xp, donation, deposit)
if deposit:
    d_token -= fee * d_token // FEE_PRECISION + 1
else:
    d_token += fee * d_token // FEE_PRECISION + 1
```

6.12 `_calc_withdraw_one_coin()` Does Not Account for NOISE_FEE

Design Low Version 3 Code Corrected

CS-CURVE-FXSWAP-V3-037

In the View contract, `_calc_withdraw_one_coin()` performs an optimization when computing fees:

```
D_fee: uint256 = fee * dD // (2 * 10**10) + 1
```

This is equivalent to the computation done by `_calc_withdraw_fixed_out()` in the Twocrypto contract:

```
# fee = sum(amounts_i - avg(amounts)) * fee' / sum(amounts)
# fee' = _fee(xp) * N_COINS / (4 * (N_COINS - 1)) = _fee(xp)/2 (for N_COINS=2)
fee: uint256 = unsafe_div(
    unsafe_mul(self._fee(xp), N_COINS),
    unsafe_mul(4, unsafe_sub(N_COINS, 1))
)

S: uint256 = 0
for _x: uint256 in amounts:
    S += _x

avg: uint256 = unsafe_div(S, N_COINS)
Sdiff: uint256 = 0

for _x: uint256 in amounts:
    if _x > avg:
        Sdiff += unsafe_sub(_x, avg)
    else:
        Sdiff += unsafe_sub(avg, _x)
```

When withdrawing only one coin (x tokens), the fee calculation simplifies as follows:

```
fee = sum(amounts_i - avg(amounts)) * fee' / sum(amounts)
fee = (|0 - x| + |x - x|) * _fee(xp)/2 / x
```

```

fee = x * _fee(xp)/2 / x
fee = _fee(xp)/2

```

However, `_calc_withdraw_one_coin()` does not account for the `NOISE_FEE` that is added by `_calc_token_fee()`:

```

return fee * Sdiff // S + NOISE_FEE + lp_spam_penalty_fee

```

Code corrected:

The fee calculation in the view contract now relies on the pool's preview function, which accounts for the noise fee.

6.13 Donation Release Can Be Artificially Delayed

Design **Low** **Version 2** **Code Corrected**

CS-CURVE-FXSWAP-V3-035

The new protection mechanism for share release updates the `last_donation_release_ts` according to the following formula when donating:

```

# We want the following equality to hold:
# self._donation_shares() = new_donation_shares * (new_elapsed / self.donation_duration)
# We can rearrange this to find the new elapsed time (imitating one large virtual donation):
# => new_elapsed = self._donation_shares() * self.donation_duration / new_donation_shares
# edge case: if self.donation_shares = 0, then self._donation_shares() is 0
# and new_elapsed = 0, thus initializing last_donation_release_ts = block.timestamp
new_elapsed: uint256 = self._donation_shares() * self.donation_duration // new_donation_shares

# Additional observations:
# new_elapsed = (old_pool * old_elapsed / D) * D / new_pool = old_elapsed * (old_pool / new_pool)
# => new_elapsed is always smaller than old_elapsed
# and self.last_donation_release_ts is carried forward proportionally to new donation size.
self.last_donation_release_ts = block.timestamp - new_elapsed

```

However, the function `_donation_shares()` is defined as follows:

```

@internal
@view
def _donation_shares() -> uint256:
    """
    @notice Calculates the amount of donation shares that are unlocked and not under protection.
    @dev This function accounts for both time-based release and add_liquidity-based protection.
    """
    donation_shares: uint256 = self.donation_shares
    if donation_shares == 0:
        return 0

    # --- Time-based release of donation shares ---
    elapsed: uint256 = block.timestamp - self.last_donation_release_ts
    unlocked_shares: uint256 = min(donation_shares, donation_shares * elapsed // self.donation_duration)

    # --- Donation protection damping factor ---
    protection_factor: uint256 = 0
    expiry: uint256 = self.donation_protection_expiry_ts
    if expiry > block.timestamp:
        protection_factor = (expiry - block.timestamp) * PRECISION // self.donation_protection_period

    return unlocked_shares * (PRECISION - protection_factor) // PRECISION

```

When protection is already active and the `protection_factor` is close to 10^{18} , the new `elapsed` calculation will be very close to 0.

An attacker can exploit this mechanism through the following steps:

Given some donation being slowly released:

1. **Activate Protection:** Add liquidity to the pool, which updates `donation_protection_expiry_ts` to activate protection for a certain period
2. **Manipulate Timestamp:** Donate a small amount of shares, which updates `last_donation_release_ts` to a value very close to the current block timestamp, since `_donation_shares()` returns a small value due to the high `protection_factor`

After this attack, the initial donation that was available for release will be delayed, as the `last_donation_release_ts` will be reset very close to the current block timestamp.

Code corrected:

In **Version 3**, `add_liquidity` was modified to use `_donation_shares(False)`, which does not apply the protection factor:

```
@internal
@view
def _donation_shares(_donation_protection: bool = True) -> uint256:
    """
    @notice Calculates the amount of donation shares that are unlocked and not under protection.
    @dev This function accounts for both time-based release and add_liquidity-based protection.
    """
    donation_shares: uint256 = self.donation_shares
    if donation_shares == 0:
        return 0

    # --- Time-based release of donation shares ---
    elapsed: uint256 = block.timestamp - self.last_donation_release_ts
    unlocked_shares: uint256 = min(donation_shares, donation_shares * elapsed // self.donation_duration)

    if not _donation_protection:
        # if donation protection is disabled, return the total amount of unlocked donation shares
        # this is needed to calculate new timestamp for overlapping donations in add_liquidity
        # otherwise must always be called with donation_protection=True
        return unlocked_shares

    # --- Donation protection damping factor ---
    protection_factor: uint256 = 0
    expiry: uint256 = self.donation_protection_expiry_ts
    if expiry > block.timestamp:
        protection_factor = (expiry - block.timestamp) * PRECISION // self.donation_protection_period

    return unlocked_shares * (PRECISION - protection_factor) // PRECISION
```

6.14 Incorrect Approximate Fee

Correctness **Low** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-005

In `_remove_liquidity_fixed_out()`, the following event is emitted:

```
log RemoveLiquidityImbalance(
    provider=msg.sender,
    lp_token_amount=token_amount,
    token_amounts=token_amounts,
```

```

    approx_fee=approx_fee * token_amount // PRECISION,
    price_scale=price_scale
)

```

However, `approx_fee` is scaled with 10^{10} and not `PRECISION`, which is 10^{18} . This leads to an incorrect fee approximation being emitted in the event.

Moreover, to be consistent with the other use of `approx_fee`, it should be rounded up with one wei, as it is done in the `_calc_withdraw_fixed_out()` function.

Code Corrected:

`approx_fee` is now divided by 10^{10} and rounded up with one wei.

6.15 Incorrect Fee Formula

Correctness **Low** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-006

The `_fee()` function in `Twocrypto` uses the following formula to calculate the fee:

```

@internal
@view
def _fee(xp: uint256[N_COINS]) -> uint256:

    # unpack mid_fee, out_fee, fee_gamma
    fee_params: uint256[3] = self._unpack_3(self.packed_fee_params)

    # warm up variable with sum of balances
    B: uint256 = xp[0] + xp[1]

    # balance indicator that goes from 10**18 (perfect pool balance) to 0 (very imbalanced, 100:1 and worse)
    # N^N * (xp[0] * xp[1]) / (xp[0] + xp[1])**2
    B = PRECISION * N_COINS**N_COINS * xp[0] // B * xp[1] // B

    # regulate slope using fee_gamma
    # fee_gamma * balance_term / (fee_gamma * balance_term + 1 - balance_term)
    B = fee_params[2] * B // (unsafe_div(fee_params[2] * B, 10**18) + 10**18 - B)

    # mid_fee * B + out_fee * (1 - B)
    return unsafe_div(fee_params[0] * B + fee_params[1] * (10**18 - B), 10**18)

```

As opposed, the `_fee()` function in `TwocryptoView` uses the following formula:

```

@internal
@view
def _fee(xp: uint256[N_COINS], swap: address) -> uint256:

    packed_fee_params: uint256 = staticcall Curve(swap).packed_fee_params()
    fee_params: uint256[3] = self._unpack_3(packed_fee_params)
    f: uint256 = xp[0] + xp[1]
    f = fee_params[2] * 10**18 // (
        fee_params[2] + 10**18 -
        (10**18 * N_COINS**N_COINS) * xp[0] // f * xp[1] // f
    )

    return (fee_params[0] * f + fee_params[1] * (10**18 - f)) // 10**18

```

Since the two formulas are not equivalent, any function of `TwocryptoView` depending on `_fee()` will return incorrect results.

Code corrected:

The formula in TwocryptoView was corrected to match the one in Twocrypto.

6.16 Incorrect MIN_A and MAX_A Values

Correctness **Low** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-007

In Twocrypto, MIN_A and MAX_A are defined as follows:

```
MIN_A: constant(uint256) = N_COINS**N_COINS * A_MULTIPLIER
MAX_A: constant(uint256) = N_COINS**N_COINS * A_MULTIPLIER * 1000
```

However, in the context of the StableswapMath contract, A is expected to be given as $\text{TRUE_A} * N^{(N_COINS - 1)} * A_MULTIPLIER$, where TRUE_A is the actual value of A as defined in the invariant. The bounds should hence account for A not being scaled by $N_COINS^{**}N_COINS$ but rather by $N_COINS^{(N_COINS - 1)}$.

Code corrected:

MIN_A was adjusted to $N_COINS * A_MULTIPLIER$ (i.e. $N_COINS^{(N_COINS-1)} * A_MULTIPLIER$ for $N_COINS = 2$), matching the $\text{TRUE_A} * N^{(N_COINS-1)} * A_MULTIPLIER$ scaling expected by StableswapMath. MAX_A was set to $10_000 * A_MULTIPLIER$, the same upper bound as in stableswap.

6.17 Last Part of Ramping Can Be Extracted When Stopping Ramping

Correctness **Low** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-044

When stopping a ramping operation, the following is done:

```
@external
def stop_ramp_A_gamma():
    """
    @notice Stop Ramping A and gamma parameters immediately.
    @dev Only accessible by factory admin.
    """
    self._check_admin()

    A_gamma: uint256[2] = self._A_gamma()
    current_A_gamma: uint256 = A_gamma[0] << 128
    current_A_gamma = current_A_gamma | A_gamma[1]
    self.initial_A_gamma = current_A_gamma
    self.future_A_gamma = current_A_gamma
    self.initial_A_gamma_time = block.timestamp
    self.future_A_gamma_time = block.timestamp
```

```
# ----- Now (block.timestamp < t1) is always False, so we return saved A.
```

```
log StopRampA(current_A=A_gamma[0], current_gamma=A_gamma[1], time=block.timestamp)
```

Consider the following scenario:

- **t0**: Ramping starts, `future_A_gamma_time` is set to `t0 + ramping_duration`.
- **t1**: User calls `exchange()`, which updates `self.D` based on the current `A` and `gamma` values that we note `A1` and `gamma1`.
- **t2**: Admin stops the ramping operation with `stop_ramp_A_gamma()`. This sets `future_A_gamma_time` to `t2` and `future_A_gamma` to the current value of `A` and `gamma` that we note `A2` and `gamma2` (returned by `self._A_gamma()`).
- **t3**: User calls `exchange()` again.

Prior to **Version 6**, the `_is_ramping()` function was defined as:

```
self.future_A_gamma_time > block.timestamp
```

This meant that after stopping a ramping operation, the condition would return `False`, and the `exchange()` at time `t3` would not recompute `D`. However, the values stored in `self.future_A_gamma`, `A2` and `gamma2` were not necessarily applied to the pool yet (`self.D` was only updated with regards to `A1` and `gamma1`). Similarly to the issue described in [Last part of ramping can be extracted](#), this means that the `exchange()` call at `t3` will use an old and invalid `D` value, which was obtained at time `t1`.

To mitigate this issue, an admin should ensure that `D` was recomputed right before stopping the ramping operation (i.e. `t1 == t2`).

Code corrected:

As of **Version 6**, `_is_ramping()` is defined as:

```
self.future_A_gamma_time > self.last_timestamp
```

When stopping a ramping operation, two cases can arise:

1. `last_timestamp == block.timestamp` (`t1 == t2`)
2. `last_timestamp < block.timestamp` (`t1 < t2`)

In the first case, this means that the ramping operation was stopped at the same time as the last call to the pool, and the new `D` value was computed right before stopping the ramping operation. Hence, the `exchange()` call at `t3` will use an up-to-date `self.D` value, which was obtained at time `t1`.

In the second case, this means that `self.D` is outdated compared to the value of `self.future_A_gamma` updated by `stop_ramp_A_gamma()`. However, as `future_A_gamma_time` is set to the current block timestamp, the condition in `_is_ramping()` will return `True`, and the `exchange()` call at `t3` will recompute `D` based on `A2` and `gamma2`.

Note that this means, however, that even though the ramping operation was stopped, the pool will still perform one more ramping step, which is not necessarily a problem, but has implications such as not being able to claim admin fees right after stopping the ramping operation.

6.18 Re-initialization of a Ramping Pool

Design Low Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-038

When a pool is ramping, `_get_D()` computes the value of `D` based on the provided `A_gamma` and `xp` values:

```
@internal
@view
def _get_D(A_gamma: uint256[2], xp: uint256[N_COINS]) -> uint256:
    # Normally we need self.D, however, if A and/or gamma are ramping,
    # we need to recalculate D using the current A and gamma values.
    if self._is_ramping():
        # ongoing ramping, recalculate D
        return staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0)
    else:
        # not ramping, use self.D from storage
        return self.D
```

In `add_liquidity()`, `old_D` is computed using `_get_D()`:

```
old_D: uint256 = self._get_D(A_gamma, old_xp)
```

Issue:

If a pool was emptied while ramping, `old_D` will be non-zero because `_get_D()` recalculates `D` using the current ramping parameters.

This creates a problem: `d_token` is computed as `token_supply * D // old_D - token_supply`, which always equals zero when `token_supply` is zero (empty pool):

```
token_supply: uint256 = self.totalSupply
d_token: uint256 = 0
if old_D > 0:
    d_token = token_supply * D // old_D - token_supply
else:
    d_token = self._xcp(D, price_scale) # <----- Making initial virtual price equal to 1.
assert d_token > 0, "nothing minted"
```

The `add_liquidity()` function will always revert with the assertion `assert d_token > 0` until the ramping period ends, preventing pool re-initialization during ramping.

Code corrected:

Pools can no longer be fully emptied as some shares are minted to the pool itself at initialization, and these shares are not burned when the last user exits the pool. Hence, the pool cannot be re-initialized anymore.

6.19 View Functions for `withdraw_one_coin` Are Incorrect

Correctness Low Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-009



In the `TwocryptoView` contract, the functions `calc_withdraw_one_coin` and `calc_fee_withdraw_one_coin` can be used to preview the amount of tokens and the fee that would be incurred when withdrawing a specific token from the pool. The calculations differ from the pool's `withdraw_one_coin` function as:

- In the view function, the fee is computed on the `xp` balances prior to the withdrawal.

```
fee: uint256 = self._fee(xp, swap)
dD: uint256 = token_amount * D // token_supply

D_fee: uint256 = fee * dD // (2 * 10**10) + 1
approx_fee: uint256 = N_COINS * D_fee * xx[i] // D

D -= (dD - D_fee)

y_out: uint256[2] = staticcall math.get_y(A, gamma, xp, D, i)
dy: uint256 = (xp[i] - y_out[0]) * PRECISION // price_scale_i
xp[i] = y_out[0]
```

- In the pool's function, the fee is computed on the `xp` balances after a withdrawal omitting fees.

```
# We compute the position on the y axis after a withdrawal of dD with the constraint
# that xp_new[i] has been reduced by amountsp[i]. This is the new position on the curve
# after the withdrawal without applying fees.
y: uint256 = (staticcall MATH.get_y(A_gamma[0], A_gamma[1], xp_new, D - dD, j))[0]
amountsp[j] = xp[j] - y
xp_new[j] = y

# The only way to compute the fees is to simulate a withdrawal as we have done
# above and then rewind and apply the fees.
approx_fee: uint256 = self._calc_token_fee(amountsp, xp_new)
dD -= dD * approx_fee // 10**10 + 1

# Same reasoning as before except now we're charging fees.
y = (staticcall MATH.get_y(A_gamma[0], A_gamma[1], xp_new, D - dD, j))[0]
# We descale y to obtain the amount dy in balances and not scaled balances.
dy: uint256 = (xp[j] - y) * PRECISION // price_scales[j]
xp_new[j] = y
```

Code corrected:

The computation in the view contract now relies on the pool's preview function.

6.20 `calc_token_fee()` Incorrect for Removal of Liquidity

Correctness **Low** **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-029

In `Twocrypto`, the function `_calc_token_fee` computes the `balance_ratio` as follows:

```
surplus_amounts: uint256[N_COINS] = amounts
if from_view:
    # When calling from the view contract no liquidity has been
    # added to the balances.
    surplus_amounts = [0, 0]
```



```
# the ratio of the balances before the liquidity operation
# balances[0] / balances[1] (adjusted for fixed precisions)
balances_ratio: uint256 = (self.balances[0] - surplus_amounts[0]) * PRECISIONS[0] * PRECISION // \
    ((self.balances[1] - surplus_amounts[1]) * PRECISIONS[1])
```

If the operation is a removal of liquidity, the balance ratio will be based on the balances after the removal—or more precisely, after the removal is simulated without fees—since `self.balances` is not updated yet, and `surplus_amounts` is the amount of tokens that would be removed from the balances:

```
# _calc_token_fee expects unscaled amounts and without decimals
# adjustments.
amounts: uint256[N_COINS] = empty(uint256[N_COINS])
amounts[i] = amount_i
if i == 0:
    amounts[1] = amountsp[1] // PRECISIONS[1] * PRECISION // price_scale
else:
    amounts[0] = amountsp[0] // PRECISIONS[0]
# The only way to compute the fees is to simulate a withdrawal as we have done
# above and then rewind and apply the fees.
approx_fee: uint256 = self._calc_token_fee(amounts, xp_new)
```

This breaks the view function `calc_token_fee`, which sets `surplus_amounts` to zero. As a result, the balance ratio is computed with the balances before the removal, which is not consistent with the actual operation. In practice, this means that the view functions will return underestimated fees for the removal of liquidity.

Code corrected:

As of Version 9, the contract has been updated to correctly compute the fees for the removal of liquidity in the view functions:

```
if from_view and deposit:
    # When calling from the view contract no liquidity has been
    # added to the balances.
    surplus_amounts = [0, 0]
```

6.21 Full Admin Fee Settings Can Make the Fresh Virtual Price Check Reject Operations

Informational **Version 9** **Code Corrected**

CS-CURVE-FXSWAP-V3-057

`_set_fee_parameters()` accepts `reserved_profit_fraction == FEE_PRECISION` and `admin_fee == FEE_PRECISION`:

```
assert reserved_profit_fraction <= FEE_PRECISION
assert admin_fee <= MAX_ADMIN_FEE
```

This is a degenerate configuration where the full reserved fee bucket is accounted to the admin. Fee-paying paths then remove the admin portion from AMM-owned balances before calling `tweak_price()`.

`tweak_price()` now compares the resulting net virtual price against a fresh pre-operation baseline:

```
assert (
    virtual_price >= vp_preop and
    (is_ramping or virtual_price >= old_virtual_price)
), "virtual price decreased"
```

When both fee parameters are set to 100%, no LP fee remains in AMM balances to offset rounding, imbalance, or shape effects from the user operation. As a result, otherwise valid fee-paying operations could possibly produce a slightly lower net-of-admin virtual price than `vp_preop` and revert with "virtual price decreased".

The issue is limited to an extreme admin-controlled configuration and was not observed in practice, but it is worth documenting because the bounds currently allow this setting and the failure mode is a pool-level liveness degradation rather than a direct loss of funds.

Code corrected:

In Version 10 of the system, the admin fee was capped to 90% to prevent such scenarios.

6.22 Ramping Can Be Started Before the Pool Is Initialized

Informational

Version 9

Code Corrected

CS-CURVE-FXSWAP-V3-060

`Twocrypto.ramp_A_gamma()` can be called while the pool has not been initialized yet and while `self.D == 0`. The function only checks that no ramp is currently active and that the new parameters are within bounds:

```
self._check_admin()
assert not self._is_ramping() # dev: "ramp active"
assert future_time > block.timestamp + MIN_RAMP_TIME - 1 # dev: "ramp time below minimum"
```

In contrast, `add_liquidity()` contains a special case for empty pools that force-stops ramping before the initial liquidity calculation:

```
if self.D == 0:
    assert not donation # dev: "donation not allowed on empty pool"
    assert self.deploy_eoa == empty(address), "!init" # also check if pool needs to be initialized
    self.future_A_gamma_time = self.last_timestamp # Finalize ramping (makes _is_ramping return False)
```

As `_is_ramping()` compares `future_A_gamma_time` against `last_timestamp`, setting both values equal makes the pool appear not to be ramping. The following `_A_gamma()` call then returns `future_A_gamma` directly, because `block.timestamp < future_A_gamma_time` is false. As a result, a ramp started before the pool is initialized is not really ramped during pool operation; it is silently truncated and the first liquidity deposit uses the final A and gamma values.

This makes the pool initialization path depend on an admin ramp that was scheduled before there was any active pool state, and requires `add_liquidity()` to carry a special-case state mutation to cancel that ramp. A clearer invariant would be to prevent ramping before the pool is initialized and has liquidity, instead of allowing the ramp and force-finalizing it during the first deposit.

Code corrected:



In Version 10 of the system, a check was added to `ramp_A_gamma()` to prevent starting a ramp before the pool is initialized.

6.23 Zero-Amount Withdrawals Skip Policy Update

Informational Version 9 Code Corrected

CS-CURVE-FXSWAP-V3-062

In `Twocrypto.remove_liquidity()`, very small LP burns can round both `withdraw_amounts` to zero. The function still reduces `self.D`, but skips `_update_policy_state()` because no token amount is transferred:

```
if withdraw_amounts[0] > 0 or withdraw_amounts[1] > 0:
    self._update_policy_state(...)
```

This allows D to change while balances and policy state remain unchanged.

Code corrected:

The check was changed to:

```
if amount > 0 and self.POLICY != empty(Policy):
    ...
```

6.24 Unsafe Use of `unsafe_div`

Informational Version 8 Code Corrected

CS-CURVE-FXSWAP-V3-066

In `add_liquidity`, the following assertion is commented as preventing a `unsafe_div` from being a division by zero:

```
assert d_token > 0, "nothing minted"
# this assert is reused for unsafe_div later
```

However, later in the function, and before the `unsafe_div` is called, `d_token` is modified:

```
d_token -= d_token_fee
```

This means that the assertion no longer guarantees that `d_token` is greater than zero at the point of the `unsafe_div`, potentially leading to a division by zero error:

```
relative_lp_add: uint256 = unsafe_div(d_token * PRECISION, token_supply + d_token) #d_token > 0
```

Although in practice, no way to reach this division with `token_supply + d_token == 0` was found, the code as written does not prevent it from happening, and is therefore more fragile to future changes, and harder to reason about.

Code corrected:

The division was changed to a safe division.

6.25 Protection Active When No Shares

Informational Version 2 Code Corrected

CS-CURVE-FXSWAP-V3-033

In `add_liquidity`, the `donation_protection_expiry_ts` is always updated, even if there are currently no active donations:

```
if relative_lp_add > 0: # sub-precision additions are expensive to stack
    # Extend protection period
    protection_period: uint256 = self.donation_protection_period
    extension_seconds: uint256 = min(relative_lp_add * protection_period // self.donation_protection_lp_threshold, protection_period)
    current_expiry: uint256 = max(self.donation_protection_expiry_ts, block.timestamp)
    new_expiry: uint256 = min(current_expiry + extension_seconds, block.timestamp + protection_period)
    self.donation_protection_expiry_ts = new_expiry
```

This means that even when no donations are available, gas is spent to update the protection, and the griefing vector described in issue [Griefing attack on liquidity additions](#) is present, even though not necessary.

Code corrected:

The issue was resolved by adding a second condition `self.donation_shares > 0` to the `if` statement above.

6.26 donation_protection_expiry_ts Not Reset

Informational Version 2 Code Corrected

CS-CURVE-FXSWAP-V3-034

When the last user leaves the pool, all important state variables are reset in `_withdraw_leftover_donations()`. However, `donation_protection_expiry_ts` is not included in this reset process.

When users add liquidity again to an empty pool, they might be incorrectly charged a `lp_spam_penalty_fee` if the `donation_protection_expiry_ts` was not zero, even though there are no donations remaining in the pool.

Code corrected:

`donation_protection_expiry_ts` is now reset to zero in `_withdraw_leftover_donations()`. The function was later removed from the codebase as pools can no longer be fully emptied.

6.27 Donation Sandwiching Through Pool Re-initialization

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-039



If the pool only has one user, the user can sandwich an incoming donation by:

1. **Exiting the pool** using `remove_liquidity`, which effectively leaves the pool empty.
2. **Allowing donation processing** which will re-initialize the pool, but not be counted as a donation, rather as a regular liquidity addition with `receiver` set to the address zero:

```
if old_D > 0:
    ...
    if donation:
        assert receiver == empty(address), "nonzero receiver"
    ...
else:

    # (re)instantiating an empty pool:

    self.D = D
    self.virtual_price = 10**18
    self.xcp_profit = 10**18
    self.xcp_profit_a = 10**18

    self.mint(receiver, d_token)
```

3. **Re-joining the pool** by adding liquidity using `add_liquidity`.

While the user does not profit from this attack, and even partially loses the donation that would have helped with future rebalancings, they are able to grief the system by:

- Burning the LP tokens of the donation
- Preventing the donation from being used for pool rebalancing
- Disrupting the intended donation mechanism

Code corrected:

The pool can no longer be re-initialized, as some shares are minted to the pool itself at initialization, and these shares are not burned when the last user exits the pool.

6.28 Function Mutability Too Broad

Informational **Version 1** **Code Corrected**

CS-CURVE-FXSWAP-V3-014

The following functions are declared as `view` but do not read the environment, they could be declared as `pure` instead:

1. `StableswapMath.get_p()`
2. `StableswapMath.newton_D()`
3. `TwocryptoView._unpack_3()`

Code corrected:

The functions have been updated to be declared as `pure`.

6.29 Incomplete and Unclear Event

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-017

In `Twocrypto.add_liquidity()`, the event `AddLiquidity` does not reflect whether the action is a donation or not. The same event also logs the LP token receiver as the provider, which can be misleading, as the receiver is not the one providing liquidity, and in case of a donation, the receiver is not the one receiving the LP tokens.

Code Corrected:

The event was adjusted to emit both the receiver and the provider in any case. Moreover, a new `Donation` event is emitted when the `add_liquidity` call is a donation.

6.30 Inconsistent approx_fee Unit

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-018

The unit of `approx_fee` in `Twocrypto._calc_withdraw_fixed_out()` and its counterpart in `TwocryptoView._calc_withdraw_one_coin()` are inconsistent.

In the `Twocrypto` contract it is defined as:

```
approx_fee: uint256 = self._calc_token_fee(amountsp, xp_new)
```

whereas the `TwocryptoView` contract defines it as:

```
approx_fee: uint256 = N_COINS * D_fee * xx[i] // D
```

The former is expressed as a percentage, while the second one is expressed as an amount of token i. `_calc_withdraw_fixed_out` is used by `_remove_liquidity_fixed_out` which logs the fee in the event as an amount of LP.

Code corrected:

`_calc_withdraw_one_coin` was removed from the system.

6.31 Misleading Variable Name

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-020

In `Twocrypto.ramp_A_gamma`, the `gamma` change is compared to `MAX_A_CHANGE`, which is misleading as it is not a change in `A` but in `gamma`.

```
ratio = 10**18 * future_gamma // A_gamma[1]
assert ratio < 10**18 * MAX_A_CHANGE + 1, "gamma change too high"
assert ratio > 10**18 // MAX_A_CHANGE - 1, "gamma change too low"
```

Code corrected:

The variable name MAX_A_CHANGE has been changed to MAX_PARAM_CHANGE.

6.32 Re-initialization of Pool Zeroes Out Admin Fees

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-040

When re-initializing the pool, unclaimed admin fees are zeroed out:

```
else:

    # (re)instantiating an empty pool:

    self.D = D
    self.virtual_price = 10**1
    self.xcp_profit = 10**18
    self.xcp_profit_a = 10**18
```

This behavior creates a potential for admin fee loss when a user is alone in the pool with a large amount of unclaimed admin fees. The user could:

1. Empty the pool by withdrawing all liquidity using `remove_liquidity`
2. Re-initialize the pool by adding new liquidity
3. Effectively zero out the admin fees and extract all accumulated profits

Code corrected:

The pool can no longer be re-initialized, as some shares are minted to the pool itself at initialization, and these shares are not burned when the last user exits the pool.

6.33 `_calc_dtoken_nofee` Does Not Handle First Deposit

Informational Version 1 Code Corrected

CS-CURVE-FXSWAP-V3-026

The function `_calc_dtoken_nofee` defined in the `TwocryptoView` contract does not handle the case where `D0 == 0` and the deposit is the first deposit of the pool as in that case, `d_token` should be computed as `self._xcp(D, price_scale)`.

Code corrected:

Client now handle the first deposit case, by relying on `self._xcp(D, price_scale)` when `D0 == 0`.

6.34 wad_exp Implicit Cast

Informational

Version 1

Code Corrected

CS-CURVE-FXSWAP-V3-027

In StableswapMath, the function `wad_exp` returns a signed `int256` value, which should always be non-negative.

The Twocrypto contract defines the following interface:

```
interface Math:
  def wad_exp(_power: int256) -> uint256: view
  ...
```

Although in practice `wad_exp()` will always return positive values, which are encoded identically in both signed and unsigned integers according to the 2's complement representation, the implicit cast from `int256` to `uint256` should be made explicit to avoid confusion and potential issues in the future.

Code corrected:

The function `wad_exp()` in the `StableswapMath` contract has been modified to explicitly cast the return value to `uint256`.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Additional Gas Savings in Twocrypto Hot Paths

Informational **Version 9** **Code Partially Corrected**

CS-CURVE-FXSWAP-V3-052

The latest Twocrypto implementation contains several additional opportunities to avoid storage reads or repeated calculations in user-facing paths.

1. In `add_liquidity()`, `vp_preop` is initialized from `self.virtual_price` before checking `old_D`. On the first-liquidity path where `old_D == 0`, this value is not used because the function does not call `tweak_price()`. The storage read can be avoided on that path.
2. In the first-liquidity branch of `add_liquidity()`, the contract writes `D`, `virtual_price`, `xcp_profit`, and `lp_xcp_profit` to storage and then immediately calls `_update_policy_state()` with several of the same values loaded back from storage:

```
self.D = D
self.virtual_price = 10**18
self.xcp_profit = 10**18
...
self._update_policy_state(..., self.virtual_price, self.xcp_profit, self.D, True)
```

These arguments are known locally in the branch and do not need to be reloaded.

3. In `remove_liquidity()`, the arguments to `_update_policy_state()` are evaluated even when no policy contract is configured. This includes recomputing `_xp(self.balances, price_scale)` and reading cached oracle, price, profit, and `D` values, while `_update_policy_state()` later returns immediately if `self.POLICY` is empty. Checking the policy before constructing the callback arguments would avoid this work in the no-policy case.
4. The same `remove_liquidity()` callback reloads `self.balances` after `_transfer_out()` updates storage and reloads `self.D` after the function has already computed the new `D` value. Keeping the post-withdraw balances and new `D` in locals would avoid those storage reads before the optional policy callback.
5. `_remove_liquidity_fixed_out()` calls `_calc_withdraw_fixed_out()`, whose setup already loads balances, computes `xp`, and computes the current ramp-aware `D`. The caller then recomputes the pre-operation invariant for `vp_preop`:

```
D_preop: uint256 = self._get_D(A_gamma, self._xp(self.balances, price_scale_preop))
vp_preop: uint256 = 10**18 * self._xcp(D_preop, price_scale_preop) // self.totalSupply
```

Returning or otherwise reusing the pre-operation `D` from the helper would avoid duplicating this calculation.

6. In `_remove_liquidity_fixed_out()`, `d_token_fee` is computed with a mandatory `+ 1`:

```
d_token_fee: uint256 = approx_fee * token_amount // FEE_PRECISION + 1
```

The subsequent `d_token_fee > 0` branch condition is therefore always true. Only the `reserved_profit_fraction` and `admin_fee` checks affect whether the branch body can run.

7. In `TwocryptoPolicy.get_price_scale()`, the policy unpacks the pool's rebalancing parameters before checking whether the cached pool state needs an EMA update:

```
rebalancing_params: uint256[3] = self._unpack_3(
    staticcall IPool(P00L).packed_rebalancing_params()
)

if state.ts < block.timestamp and price_scale > 0:
    ...
```

`rebalancing_params` is only used inside that branch. Moving the `staticcall` and unpacking into the branch would avoid the external read on calls that return the cached `price_scale` directly.

8. In `TwocryptoView._calc_dtoken_nofee()`, the helper returns `amountsp`:

```
return d_token, amountsp, xp
```

The two callers, `calc_token_amount()` and `calc_fee_token_amount()`, ignore the returned `amountsp` and only use `d_token` and `xp`. Removing it from the helper return values would avoid building and copying an unused array in the view path.

Code partially corrected:

Only item 3 was implemented.

7.2 Admin Can Disable Rebalancing by Setting Both Adjustment Steps to Zero

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-053

`apply_new_parameters()` accepts the new `adjustment_step_min` and `adjustment_step_max` without enforcing a strictly-positive lower bound:

```
# Twocrypto.vy::apply_new_parameters
if new_adjustment_step_min > 10**18:
    new_adjustment_step_min = current_rebalancing_params[0]
if new_adjustment_step_max > 10**18:
    new_adjustment_step_max = current_rebalancing_params[1]
...
assert new_adjustment_step_min <= new_adjustment_step_max
```

The only constraints are `min <= max` and that values above `10**18` are silently substituted with the previous setting. `(0, 0)` is a valid input pair.

In `tweak_price()` the rebalance gate uses these values:

```
adjustment_step: uint256 = min(
    unsafe_div(norm, 5), rebalancing_params[1] # adjustment_step_max
```

```
)  
if adjustment_step > rebalancing_params[0]:      # adjustment_step_min  
    ...                                           # commit rebalance step
```

With `adjustment_step_max == 0` the `min(norm/5, 0)` evaluates to `0`, and `0 > 0` is false, so no rebalance step is ever taken. With only `adjustment_step_min == 0` (and `max > 0`), rebalancing still works, but the floor is removed and dust-sized rebalances become permitted.

Acknowledged:

Curve acknowledges the informational finding.

7.3 Constructor Enforces Bounds on an Unused gamma Parameter

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-054

The `__init__` constructor of `Twocrypto.vy` validates the `gamma` value against `MIN_GAMMA` / `MAX_GAMMA` even though `gamma` is only used as a magic value for the allow list.

Acknowledged:

Curve acknowledges the informational finding.

7.4 Constructor Sets Fee Parameters Without Setter-Equivalent Bounds

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-055

The `__init__` constructor of `Twocrypto.vy` initialises `reserved_profit_fraction` and `admin_fee` directly with hardcoded literal values, bypassing the bounds checks that `_set_fee_parameters` applies on every subsequent update.

Acknowledged:

Curve acknowledges the informational finding.

7.5 Donation Parameters Apply Retroactively to In-Flight Schedules

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-056

`set_donation_parameters()` writes `donation_duration` and `donation_protection_period` directly to storage without rebasing already-running unlock or protection schedules:

```
self.donation_duration = duration
self.donation_protection_period = protection_period
```

Both parameters appear as divisors in formulas that compare storage-based timestamps against the live `block.timestamp`:

```
# _donation_shares()
elapsed: uint256 = block.timestamp - self.last_donation_release_ts
unlocked_shares: uint256 = min(
    donation_shares,
    unsafe_div(donation_shares * elapsed, self.donation_duration),
)
```

```
# _donation_shares() / _fee()
protection_factor = min(
    unsafe_div(unsafe_sub(expiry, block.timestamp) * PRECISION,
                self.donation_protection_period),
    PRECISION,
)
```

`last_donation_release_ts` and `donation_protection_expiry_ts` were both anchored under the *previous* values of these parameters. When the admin updates the parameter, every in-flight donation and every in-flight protection window is silently re-rated against the new divisor.

Acknowledged:

Curve acknowledges the informational finding.

7.6 Donation Path Does Not Credit `d_token_fee` to Admin Balances

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-069

In `Twocrypto.add_liquidity()`, `d_token_fee` is computed and subtracted from `d_token` on both the donation and the regular deposit branch:

```
d_token_fee: uint256 = 0
if old_D > 0:
    d_token_fee = (
        self._calc_token_fee(amounts_received, xp, donation, True) * d_token // FEE_PRECISION + 1
    ) # for donations - we only take NOISE_FEE (check _calc_token_fee)
    d_token -= d_token_fee

if not donation:
    if d_token_fee > 0 and self.reserved_profit_fraction > 0 and self.admin_fee > 0:
        # Convert the admin's share of the LP haircut into a pro-rata
        # token balance using the no-fee supply basis.
```

```

    fee_supply: uint256 = token_supply + d_token + d_token_fee
    local_balances: uint256[N_COINS] = self._apply_admin_d_token_fee(
        balances,
        d_token_fee,
        fee_supply,
    )
    xp = self._xp(local_balances, price_scale)
    D = staticcall MATH.newton_D(A_gamma[0], A_gamma[1], xp, 0)

    if donation:
        ...

```

For regular deposits the admin share of `d_token_fee` is moved from `self.balances` to `self.admin_balances` via `_apply_admin_d_token_fee()`. On the donation branch the equivalent call is omitted, so the fee charged on donations is implicitly absorbed by existing LPs (the donor receives fewer `donation_shares` than the gross `d_token` would have implied, while `self.balances` is unchanged) and never reaches `self.admin_balances`.

The amount involved is small in practice because `_calc_token_fee()` returns only `NOISE_FEE` for donations:

```

# Donation fees are 0, but NOISE_FEE is required for numerical stability
return NOISE_FEE

```

so a typical donation forfeits at most the `NOISE_FEE` portion plus the `+ 1` dust safety. The state reconstruction through events will be off by the accumulated value as the fee is not split like other fees between admin and LP.

Acknowledged:

Curve acknowledges the informational finding.

7.7 LP Xcp Profit Threshold Scales Down Proportionally When Virtual Price Decreases

Informational Version 9 Acknowledged

CS-CURVE-FXSWAP-V3-058

When `tweak_price()` observes a virtual price decrease, it decreases `xcp_profit` by the same absolute VP delta and then scales `lp_xcp_profit` down in proportion to the remaining `xcp_profit` growth:

```

elif virtual_price < old_virtual_price:
    xcp_profit -= unsafe_sub(old_virtual_price, virtual_price)
    if xcp_profit > PRECISION:
        lp_xcp_profit = PRECISION + unsafe_div(
            unsafe_sub(lp_xcp_profit, PRECISION) *
            unsafe_sub(xcp_profit, PRECISION),
            unsafe_sub(old_xcp_profit, PRECISION)
        )
    else:
        lp_xcp_profit = PRECISION

```

This means `lp_xcp_profit` is not a monotonic watermark. A ramping or curve-shape driven VP decrease reduces the LP-protected threshold by the same fraction of previously accumulated

xcp_profit growth that remains after the loss. If the loss consumes all growth above PRECISION, the threshold is reset to PRECISION.

The behavior is relevant because lp_xcp_profit gates later rebalances and donation-share burns. After a VP decrease, future rebalances can be accepted against the reduced threshold rather than the pre-loss LP threshold, so the contract preserves only the surviving profit growth instead of treating the old lp_xcp_profit value as permanently protected.

As of Version 10, the case of a VP decrease is handled differently:

```
else: # virtual_price <= old_virtual_price:
    vp_delta: uint256 = unsafe_sub(old_virtual_price, virtual_price)
    xcp_profit -= vp_delta
    if lp_xcp_profit > PRECISION and vp_delta <= unsafe_sub(lp_xcp_profit, PRECISION):
        lp_xcp_profit = unsafe_sub(lp_xcp_profit, vp_delta)
    else:
        lp_xcp_profit = PRECISION
```

Instead of scaling the LP threshold proportionally to the surviving xcp_profit growth, Version 10 subtracts the full virtual-price loss from lp_xcp_profit and only floors the result at PRECISION. This makes the threshold decay point-for-point with ramping or curve-shape losses, regardless of how much of the historical xcp_profit growth had been allocated to the LP-protected bucket. The practical implication is that a VP decrease consumes the LP rebalance floor faster than the older proportional rule in the common case where lp_xcp_profit - PRECISION is smaller than xcp_profit - PRECISION; once the absolute loss reaches the protected surplus, later rebalances are gated only by the baseline PRECISION floor.

Acknowledged:

Curve acknowledges the informational finding.

7.8 Magic-gamma Initialization Records Tx.Origin

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-059

When gamma_A[0] is the magic initialization value, Twocrypto.vy stores deploy_eoa from tx.origin. initialize() then permits the one-time setup only when msg.sender is that stored address or the factory admin.

This makes the path incompatible with smart-contract wallets and meta-transaction flows: the contract wallet cannot initialize the pool itself, and a relayer or bundler EOA may receive the deployer initialization right instead of the user.

Acknowledged:

Curve acknowledges the informational finding.

7.9 Policy State Can Receive Stale D During Ramping

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-070

During an active A or gamma ramp, `self.D` can lag the live `_A_gamma()` values because it is only refreshed by operations that recompute the invariant. Two paths can still push that cached D to the external policy:

```
# remove_liquidity()
D: uint256 = self.D # no ramping adjustment to preserve safety of balanced removal
...
self._update_policy_state(..., self.D, False)

# set_policy_contract()
self._update_policy_state(
    ...,
    self.D,
    True,
)
```

Balanced withdrawals do this by design: they avoid invariant math and scale the cached D down proportionally. `set_policy_contract()` similarly initializes the newly attached policy from cached pool state. In both cases, the policy can receive a `last_pool_state.D` that is stale with respect to the current ramped A and gamma values used by rebalancing logic.

The bundled `TwocryptoPolicy` does not currently use D in `get_fee()` or `get_price_scale()`. However, the policy interface is external. If a custom policy uses D to size fees, choose a target price, or otherwise guide rebalancing, these paths can make it act on outdated invariant data until a later state-changing operation refreshes policy state.

Acknowledged:

Curve acknowledges the informational finding.

7.10 View Functions Return Stale D During Ramping

Informational **Version 9** **Acknowledged**

CS-CURVE-FXSWAP-V3-061

Several view-only entry points read `self.D` directly instead of recomputing a ramp-aware D via `MATH.newton_D` or `TwocryptoView._calc_D_ramp`. While A or gamma are ramping, `self.D` is the value committed at the previous `tweak_price()` call and is no longer the invariant that matches the live (A, gamma) pair.

The following entry points are affected: `lp_price()` and `get_virtual_price()`.

Acknowledged:

Curve acknowledges the informational finding.

7.11 TokenExchange Coin-Id Fields Are Not Indexed

Informational Version 9 Acknowledged

CS-CURVE-FXSWAP-V3-063

The TokenExchange event in Twocrypto.vy only marks buyer as indexed. sold_id and bought_id are left as plain uint256 fields even though both are constrained to {0, 1} (the pool has N_COINS == 2) and would therefore be cheap to filter on.

Off-chain consumers that want per-direction analytics (e.g. tracking swaps from coin 0 to coin 1 separately) currently have to load the full event payload and filter client-side, where indexing the two coin-id fields would let them subscribe directly via topic filters.

Acknowledged:

Curve acknowledges the informational finding.

7.12 Donations Can Revert While Released Shares Remain Unburned

Informational Version 8 Acknowledged

CS-CURVE-FXSWAP-V3-064

Integrators that route external revenue, MEV proceeds, or other automated payments through donation deposits should treat donation calls as fallible.

When Twocrypto.add_liquidity() is called with donation=True, the pool checks the donation cap against the stored self.donation_shares value plus the newly minted shares:

```
new_donation_shares: uint256 = self.donation_shares + d_token
assert new_donation_shares * PRECISION // (token_supply + d_token) <= self.donation_shares_max_ratio, "donation above cap!"
```

The time-based release path in _donation_shares() computes how many donation shares are unlocked, but it does not reduce self.donation_shares storage. That storage value is only reduced later when tweak_price() burns available donation shares during a virtual-price-lowering adjustment.

As a result, a pool can reject new donations with donation above cap! even after earlier donations have fully time-vested. This can persist in quiet pools, or in pools whose rebalances do not lower virtual price enough to burn the released donation shares. Donation integrators should therefore avoid assuming that elapsed time alone reopens donation capacity, and should handle failed donation attempts without blocking the rest of their revenue or accounting flow.

Acknowledged:

Curve acknowledges the informational finding.

7.13 Dust Donations Can Revert After the Pre-Fee Mint Check

Informational Version 8 Acknowledged

In `Twocrypto.add_liquidity()`, donation LP shares are first checked with `assert d_token > 0` and then reduced by the donation noise fee:

```
d_token_fee = self._calc_token_fee(amounts_received, xp, donation, True) * d_token // FEE_PRECISION + 1
d_token -= d_token_fee
```

For a donation that would mint exactly one LP unit before fees, the `+ 1` fee reduces `d_token` to zero. If there are no existing donation shares, the subsequent `new_elapsed` calculation divides by `new_donation_shares == 0` and reverts.

Integrators should not rely on dust-sized donation calls as a successful no-op donations. Donation previews or callers should treat post-fee zero-share donations as reverting inputs.

Acknowledged:

Curve acknowledges the informational finding.

7.14 Exchange Fee Rounds Down

Informational **Version 8** **Acknowledged**

CS-CURVE-FXSWAP-V3-065

In `Twocrypto._exchange()`, the swap fee is computed with `unsafe_div(self._fee(xp) * dy, FEE_PRECISION)`, which floors the result. The LP fee and derived admin fee therefore round down, leaving sub-unit fee dust with the trader.

This slightly disfavors the pool and is inconsistent with nearby fee paths that add `+ 1` so rounding dust accrues to the pool.

Acknowledged:

Curve acknowledges the informational finding.

7.15 `find_pool_for_coins` Can Return Pools for Colliding Coin Pairs

Informational **Version 8** **Acknowledged**

CS-CURVE-FXSWAP-V3-067

`TwocryptoFactory` indexes pools by XORing the two coin addresses:

```
key: uint256 = convert(coin_a, uint256) ^ convert(coin_b, uint256)
self.markets[key].append(pool)
```

`find_pool_for_coins()` uses the same key and returns the indexed pool without checking that the returned pool actually contains `_from` and `_to`:

```
key: uint256 = convert(_from, uint256) ^ convert(_to, uint256)
return self.markets[key][i]
```

XOR is not unique across address pairs, so two distinct pairs can share the same market key. If such a collision exists, `find_pool_for_coins()` may return a pool for a different pair than the one requested.

This is unlikely to be practically exploitable because producing a useful collision between real ERC20 contract addresses is computationally hard under normal deployment assumptions. However, it can break off-chain or on-chain integrators that rely on `find_pool_for_coins()` alone and do not re-check the returned pool's coins with `get_coins()` or `get_coin_indices()`.

Acknowledged:

Curve acknowledges the informational finding.

7.16 Rebalancing DOS Attack

Informational **Version 7** **Acknowledged**

CS-CURVE-FXSWAP-V3-045

In Version 7 of the system, the `tweak_price` function has been updated to allow at most one rebalancing per block. This rebalancing must occur during the first call to the pool in a given block. This means that, if a malicious party wanted to prevent rebalancing in a pool, they could submit a transaction in every block that interacts with the pool before any other user. That transaction would put the pool in a state where rebalancing would not succeed, before undoing the changes to avoid arbitrage. However, this would require significant resources, and the incentives are unclear, as the attacker would not gain any direct profit from this action.

Acknowledged:

Curve acknowledges this issue.

7.17 Fees Are Based on Balances After Removal of Liquidity

Informational **Version 2** **Acknowledged**

CS-CURVE-FXSWAP-V3-031

In Twocrypto, the function `_calc_token_fee` computes the `balance_ratio` as follows:

```
balance_ratio: uint256 = (self.balances[0] - surplus_amounts[0]) * PRECISIONS[0] * PRECISION // ((self.balances[1] - surplus_amounts[1]) * PRECISIONS[1])
```

If the operation is a removal of liquidity, the balance ratio will be based on the balances after the removal—or more precisely, after the removal is simulated without fees—since `self.balances` is not updated yet, and `surplus_amounts` is the amount of tokens that would be removed from the balances:

```
# _calc_token_fee expects unscaled amounts and without decimals
# adjustments.
amounts: uint256[N_COINS] = empty(uint256[N_COINS])
amounts[i] = amount_i
if i == 0:
    amounts[1] = amountsp[1] // PRECISIONS[1] * PRECISION // price_scale
else:
    amounts[0] = amountsp[0] // PRECISIONS[0]
# The only way to compute the fees is to simulate a withdrawal as we have done
```

```
# above and then rewind and apply the fees.
approx_fee: uint256 = self._calc_token_fee(amounts, xp_new)
```

This means that for a removal of liquidity that is not balanced given the current pool's balances, the fee incurred will be higher than it would if the ratio was computed based on the balances before the liquidity operation, as is done when adding liquidity.

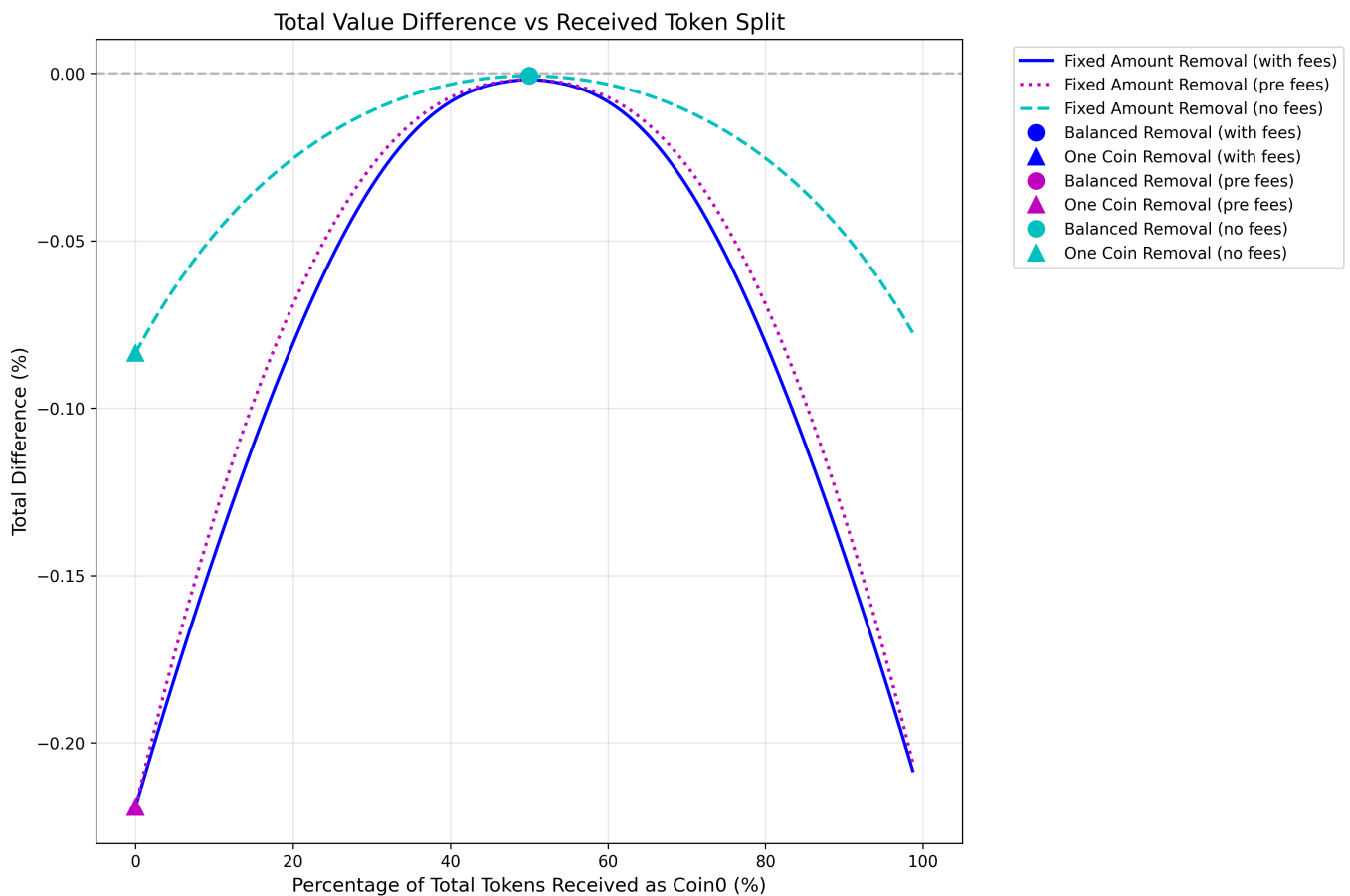
Example:

- The pool is initialized with 100,000 coin A and 100,000 coin B.
- A user adds 50,000 coin A and 50,000 coin B (worth a total of 100,000 coin A given a price of 1:1) and obtains X LP tokens.

The graph below shows the loss of value for the user after removing liquidity using the X LP tokens, using different methods and splits of tokens to receive.

- **Y axis:**

$$(\text{coinA_out} + \text{coinB_out} - \text{coinA_in} - \text{coinB_in}) / (\text{coinA_in} + \text{coinB_in})$$
- **X axis:** $\text{coin0_out} / (\text{coin0_out} + \text{coin1_out})$



Acknowledged:

The mismatching fee calculation was acknowledged by Curve.

7.18 Protection Expiry Can Become Smaller Than the Current One

Informational Version 2 Acknowledged

CS-CURVE-FXSWAP-V3-041

If the admin does not carefully set the new `donation_protection_period`, it is possible that in `add_liquidity()`, the `new_expiry` becomes smaller than the `current_expiry`:

```
new_expiry: uint256 = min(current_expiry + extension_seconds, block.timestamp + protection_period)
```

Explanation:

The first part of the expression (`current_expiry + extension_seconds`) will always be greater than `current_expiry`. However, `block.timestamp + protection_period` might be smaller if `self.donation_protection_period` was sufficiently decreased by the admin beforehand.

This means the `min()` function could select the smaller value, resulting in a new expiry that is actually earlier than the current one.

Acknowledged:

Curve acknowledged the behavior.

7.19 Code Style Inconsistencies

Informational Version 1 Code Partially Corrected

CS-CURVE-FXSWAP-V3-010

The code style in the contract does not follow the recommended conventions, which can lead to reduced readability and maintainability. The following issues were identified:

1. `@internal` functions are not consistently prefixed with the `_` underscore prefix. E.g. `tweak_price()`.
2. Parameters in functions are not consistently named. They are sometimes named with a leading underscore and sometimes without.
3. TODO comments are left in code: `# TODO improve comments here`
4. In `StableswapMath`, the following comment could be removed:

```
# MIN_GAMMA: constant(uint256) = 10**10
# MAX_GAMMA_SMALL: constant(uint256) = 2 * 10**16
# MAX_GAMMA: constant(uint256) = 199 * 10**15 # 1.99 * 10**17
```

5. In `StableswapMath`, the function `get_y()` takes `A` as a parameter, while the function `newton_D` takes `_amp`. As these are the same value, it would be better to use the same name for both parameters to avoid confusion.
6. In `StableswapMath`, the function `newton_D` fails with `raise "Did not converge"` if Newton's method does not converge. `get_y()` only raises but without a message. It would be better to raise with a message in both cases for consistency.
7. In `StableswapMath`, `pow_mod256(N_COINS, N_COINS)` could be replaced with `N_COINS ** N_COINS` for better readability.

8. In several occurrences, loops are used with a range of `N_COINS` to iterate over a single array with `arr[i]`. It would be more readable to iterate directly over the array instead of using a range:

- In `StableswapMath.get_p()`:

```
for i: uint256 in range(N_COINS):  
    Dr = Dr * _D // _xp[i]
```

- In `TwocryptoFactory.deployPool()`:

```
for i: uint256 in range(N_COINS):  
    d: uint256 = convert(staticcall IERC20(_coins[i]).decimals(), uint256)  
    assert d < 19, "max 18 decimals for coins"  
    decimals[i] = d  
    precisions[i] = 10 ** (18 - d)
```

9. In multiple functions of the `TwocryptoView` contract, the swap parameter is passed around as an address and is cast to `Curve` before each call. It would be more readable to cast it to the `Curve` interface once at the beginning of the function or directly have the external functions take a `Curve` interface as parameter instead of an address.
10. In `Twocrypto`, `remove_liquidity_one_coin` used the parameter `lp_token_amount` which is not consistent with the functions `remove_liquidity_fixed_out` and `_remove_liquidity_fixed_out` which use `token_amount`. It would be better to use the same name for both parameters to avoid confusion.
11. In the different packing and un-packing functions, the magic value `18446744073709551615` (`2 ** 64 - 1`) is used for masking integers. It would be more readable to define it in a constant variable, e.g. `MAX_UINT64: constant(uint256) = 2 ** 64 - 1` and use it instead.
12. In `Twocrypto._exchange()`, if `self._is_ramping()`, the function manually recomputes the old `D`. It would be more readable to use the functions `_xp()` and `_get_D()` as it is done in other functions.
13. In `Twocrypto._A_gamma()`, `unpack_2()` could be used to unpack the `A_gamma` variable instead of manually unpacking it.
14. In `Twocrypto.tweak_price()`, the following comment is badly formatted:

```
if vp_boosted > threshold_vp + rebalancing_params[0]:  
    # allowed_extra_profit -----^
```

15. In `TwocryptoView`, the function `_prep_calc()` defines a variable `xp` which holds raw balances, the name is misleading:

```
xp: uint256[N_COINS] = empty(uint256[N_COINS]) # @INFO this is not xp, this is x  
for k: uint256 in range(N_COINS):  
    xp[k] = staticcall Curve(swap).balances(k)
```

16. The `Math` interface in `Twocrypto.vy` uses parameter names that diverge from `StableswapMath.vy`. For example, `newton_D` and `get_y` name the amplification parameter `ANN` in the interface but `_amp` in the implementation, and `get_p` declares `_A_gamma` as `uint256[2]` while the implementation uses `uint256[N_COINS]`.

Code partially corrected:

Only items 3, 4, 5, 6, 7 and 14 were corrected. The other items will be addressed in a future version of the system.

7.20 Duplicated Code

Informational **Version 1** **Code Partially Corrected**

CS-CURVE-FXSWAP-V3-011

The system contains duplicated code that could be refactored to reduce redundancy and improve maintainability.

1. In Twocrypto, `exchange()` and `exchange_received()` perform almost identical operations. They could be refactored to a single function that takes an additional parameter `expect_optimistic_transfer`, or both call an internal function that performs the common operations.
2. `tweak_price()` and `internal_price_oracle()` in Twocrypto perform similar operations. They could be refactored to call a common internal function that performs the operations.
3. In TwocryptoView, the function `_calc_withdraw_one_coin()` performs very similar operations as `_prep_calc()`, which is usually used to prepare the calculation. The former could be refactored to call the latter.

Code partially corrected:

`_calc_withdraw_one_coin` was removed from TwocryptoView as of Version 9, for the other items, Curve answered that the duplicated code will be refactored in a later version of the system.

Note that as of Version 9, `internal_price_oracle` was moved to the TwocryptoView contract, but still performs similar operations as `tweak_price` in the main contract.

7.21 Event Fields Indexing

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-012

None of the events emitted by the TwocryptoFactory have indexed fields. Indexing fields in events allows for more efficient querying and filtering of logs by off chain applications.

Acknowledged:

The client has acknowledged this issue. But will not alter the factory contract in that version.

7.22 Factory's future_admin Not Reset

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-013

In the TwocryptoFactory, the `future_admin` variable is not reset to the zero address after the admin change is executed. This could lead to confusion as the new admin can then call `accept_transfer_ownership` again and emit `TransferOwnership` multiple times.

Acknowledged:

Curve has acknowledged this issue but will not alter the factory contract in this version.

7.23 Gas Savings

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-015

The system contains several functions that can be optimized to save gas costs. The following issues were identified:

StableswapMath:

1. In `get_y()`, the following loop will enter the `if` branch exactly once and the `else` branch exactly once, it could be inlined to save gas:

```
for _i: uint256 in range(N_COINS):
    if _i != i:
        _x = xp[_i]
    else:
        continue
    S_ += _x
    c = c * D // (_x * N_COINS)
```

2. In `newton_D()`, the following loop could be inlined to save gas:

```
for x: uint256 in _xp:
    S += x
```

Twocrypto:

1. In `_transfer_in()`, when `expect_optimistic_transfer` is set to `True`, `self.balances[_coin_idx]` is read twice from storage, which could be optimized by using a local variable.
2. `_transfer_in()` has `sender` as a parameter, but it is only called with `msg.sender`, the parameter could be removed to save gas.
3. In `add_liquidity()`, `balances` could be initialized with `old_balances` instead of `self.balances` to save gas.
4. In `add_liquidity()`, `self.totalSupply += d_token` could be simplified to use the local `token_supply` variable instead of reading from storage.
5. In `remove_liquidity()`, the following could be replaced by `withdraw_amounts = self.balances` to save gas:

```
for i: uint256 in range(N_COINS):
    withdraw_amounts[i] = self.balances[i]
```

6. In `_remove_liquidity_fixed_out()`, the `_transfer_out()` calls could be done conditionally when the token amount is non-zero to save gas on transfers of 0 tokens.
7. In `_exchange()`, when ramping, `D` is stored in storage and then immediately read back, to be latter overwritten. This could be optimized by using a local variable.

8. In `_exchange()`, `xp[j] -= dy` could be replaced with `xp[j] = y_out[0]`.
9. In `_exchange()`, the call to `MATH.newton_D()` could omit the last argument as it is not used in the function.
10. `_claim_admin_fees()` could be refactored as when not entering the branch `fee_receiver != empty(address)` and `fees > 0`, `admin_share = 0`, and in that case `total_supply_including_admin_share == current_lp_token_supply`. Similarly, the following computation could now be simplified:

```
frac: uint256 = vprice * 10**18 // (vprice - fees) - 10**18
admin_share += current_lp_token_supply * frac // 10**18
```

11. In `_calc_token_fee()`, the following could be simplified to save gas:

```
fee: uint256 = unsafe_div(
    unsafe_mul(self._fee(xp), N_COINS),
    unsafe_mul(4, unsafe_sub(N_COINS, 1))
)
```

12. Multiple parameters that are usually used together and do not need a full word could be packed to save gas. E.g., `donation_duration` and `last_donation_release_ts` or, as of [Version 2](#), the donation protection parameters.

TwocryptoFactory:

1. In `deploy_gauge()`, `gauge_implementation` is loaded twice from storage and could be cached.

Acknowledged:

Curve acknowledges the gas savings and answered that they will be implemented in a later version of the system.

7.24 Incomplete NatSpecs

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-016

The system contains functions that have incomplete NatSpec documentation, which can lead to confusion for users and developers. The following list of functions was identified:

TwocryptoFactory:

- `initialise_ownership`: param `_fee_receiver` is missing
- `initialise_ownership`: param `_admin` is missing
- `deploy_pool`: param `_coins` is missing
- `deploy_pool`: param `implementation_id` is missing
- `deploy_pool`: param `A` is missing
- `deploy_pool`: param `gamma` is missing
- `deploy_pool`: param `mid_fee` is missing
- `deploy_pool`: param `out_fee` is missing
- `deploy_pool`: param `fee_gamma` is missing

- `deploy_pool`: param `allowed_extra_profit` is missing
- `deploy_pool`: param `adjustment_step` is missing
- `deploy_pool`: param `ma_exp_time` is missing
- `deploy_pool`: param `initial_price` is missing
- `get_market_counts`: param `coin_a` is missing
- `get_market_counts`: param `coin_b` is missing

TwocryptoView:

- `get_dy`: all params are missing
- `get_dy`: return is missing
- `get_dx`: all params are missing
- `get_dx`: return is missing
- `calc_withdraw_one_coin`: all params are missing
- `calc_withdraw_one_coin`: return is missing
- `calc_token_amount`: all params are missing
- `calc_token_amount`: return is missing
- `calc_fee_get_dy`: all params are missing
- `calc_fee_get_dy`: return is missing
- `calc_fee_withdraw_one_coin`: all params are missing
- `calc_fee_withdraw_one_coin`: return is missing
- `calc_fee_token_amount`: all params are missing
- `calc_fee_token_amount`: return is missing

Twocrypto:

- `set_donation_duration`: param `duration` is missing
- `tweak_price()`: natspec mentions `new_D` instead of `D`
- `tweak_price()`: return is missing

StableswapMath:

- `get_y`: all params are missing
- `get_y`: return is missing
- `newton_D`: all params are missing
- `newton_D`: return is missing
- `get_p`: return is missing
- `wad_exp`: param `x` is missing
- `wad_exp`: return is missing

Acknowledged:

Client has acknowledged this issue and will address it in a future update.

7.25 Incorrect Comments

Informational

Version 1

Code Partially Corrected

CS-CURVE-FXSWAP-V3-019

The system contains comments that are either incorrect or misleading, which can lead to confusion when reading the code. The following issues were identified:

StableswapMath:

1. The following comment is incorrect as it computes the value of `xp[i]` given `xp[j]` as the provided parameters are already multiplied by the price scales.

```
"""  
Calculate x[i] for given x[j] (j != i) and D.  
"""
```

2. The following comment is misleading as there is no `x` parameter in `get_y()`.

```
# x in the input is converted to the same price/precision
```

3. In the function `get_p()`, the dev natspec states the following, the question mark should be removed to clarify the comment:

```
"""  
...  
Output needs to be multiplied with price_scale to get the actual value. ?  
...  
"""
```

Twocrypto:

1. The following comment is misleading:

```
last_timestamp: public(uint256)    # idx 0 is for prices, idx 1 is for xcp.
```

2. The docstrings for `internal_price_oracle()`, `price_oracle()` and `price_scale()` reference a `k` parameter that is no longer present.

3. In `_calc_withdraw_fixed_out`, the following comment mentions `D0` which does not exist:

```
# ----- Calculate D0 and xp -----
```

4. In `_calc_withdraw_fixed_out`, the following comment is incorrect as there is no such thing as adjusting `D`:

```
# We adjust D not to take into account any donated amount. Donations  
# should never be withdrawable by the LPs.
```

5. In `remove_liquidity`, the following comment is misleading as there is no such "adjusted amount":

```
# We intentionally use the unadjusted `amount` here as the amount of lp  
# tokens burnt is `amount`, regardless of the rounding error.
```

6. In `add_liquidity`, the natspec describes the following, which is not true in case of a donation:

```
"""  
...  
"""
```

```
@return uint256 Amount of LP tokens received by the `receiver`  
...  
!!!!
```

7. In `_donation_shares()`, the following comment is misleading as one of the cases should be `<=` instead of `<`:

```
# ===== absorption rate logic =====  
# `elapsed > self.donation_duration` => release whatever is left in `self.donation_shares` to avoid underflow.  
# `elapsed < self.donation_duration` => release at the current rate.
```

8. In `_donation_shares()`, the following comment is misleading as if `elapsed == 0`, the function returns 0:

```
# `elapsed == 0` => multiple absorption attempts in the same block, return current value of `self.donation_shares` in storage.
```

9. In `_exchange()`, the following comment is misleading as it is not the old `xp[i]` but the old `x[i]`:

```
x0: uint256 = balances[i] - dx_received # old xp[i]
```

10. In `apply_new_parameters()`, the following assertion's error message is misleading as `10**18` is not included in the range:

```
if new_fee_gamma < 10**18:  
    assert new_fee_gamma > 0, "fee_gamma out of range [1 .. 10**18]"
```

11. In `_exchange()`, the following comment is misleading as `newton_D()` does not use the provided initial guess:

```
# ----- Tweak price_scale with good initial guess for newton_D -----
```

12. In `tweak_price()`, the following comment is misleading:

```
if last_timestamp < block.timestamp: # 0th index is for price_oracle.  
    ...
```

13. `tweak_price()`'s natspec should mention `_remove_liquidity_fixed_out` instead of `remove_liquidity_fixed_out`.

14. In `_transfer_in()`, the following comment is misleading as the function in fact acknowledges and uses tokens donated to the pool (using ERC20 transfers):

```
# If someone donates extra tokens to the contract: do not acknowledge.  
# We only want to know if there are dx amount of tokens. Anything extra,  
# we ignore. This is why we need to check if received_amounts (which  
# accounts for coin balances of the contract) is at least dx.
```

15. In `add_liquidity()`, the following comment is misleading as admin fees are not claimed:

```
# ----- Log and claim admin fees.
```

Code partially corrected:

All the comments in `StableswapMath` and items 1, 6, 7, 8, 10 and 14 for `Twocrypto` were fixed or removed. Remaining comments will be fixed in a later version of the system.

7.26 Missing Events

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-021

The system does not emit any events related to the following important actions:

- `commit_transfer_ownership()` in `TwocryptoFactory`.
- `add_liquidity()` in `Twocrypto` does not emit an ERC20 Transfer event when the action is a donation.
- The constructor of `Twocrypto` does not emit the following events, although the corresponding values are set: `NewParameters`, `SetDonationDuration`, `SetAdminFee`. Similarly, no event is emitted for the initial values of `gamma` and `A`.

Acknowledged:

Client has acknowledged this issue and has decided not to emit events for these actions.

7.27 Missing Sanity Checks

Informational **Version 1** **Code Partially Corrected**

CS-CURVE-FXSWAP-V3-022

The system does not perform sanity checks on the input parameters of several functions:

In `Twocrypto`:

1. In `set_donation_duration()`, the function does not perform any check on the new duration.
2. In `add_liquidity()`, if `is_donation`, the function does not check that the provided receiver is the zero address.
3. In `add_liquidity()`, if the provided receiver is the zero address, the function does not check that the action is a donation.
4. In `add_liquidity()`, if `is_donation`, the function does not check that the action is not the first deposit of the pool.
5. In `add_liquidity()`, if `d_token > 0`, but `d_token - d_token_fee == 0`, the function does not revert and mints 0 LP tokens.
6. In the constructor, no sanity checks are performed on the provided fees and rebalancing parameters. Although they are checked in the factory, arbitrary values could be provided if the pool is not created through the factory.

In `TwocryptoFactory`:

1. In `initialise_ownership()`, `_admin` is not checked to be non-zero.

In `TwocryptoView`:

1. `calc_fee_token_amount()` does not check that if `is_donation` is true, `deposit` is also true.
2. `calc_token_amount()` does not check that if `is_donation` is true, `deposit` is also true.

Code partially corrected:

For Twocrypto, sanity checks were added for items 1, 2 and 4. Curve answered that item 3 will not be corrected as it is intended that users can burn their LP tokens by sending them to the zero address. Other items remain unchanged.

7.28 Missing View Function for `remove_liquidity_fixed_out`

Informational **Version 1** **Acknowledged**

CS-CURVE-FXSWAP-V3-023

In the `TwocryptoView` contract, no functions are defined to preview the result of a call to `remove_liquidity_fixed_out()` as it is done for removing liquidity in one coin only with `calc_withdraw_one_coin()` and `calc_fee_withdraw_one_coin()`.

Acknowledged:

Client has acknowledged this issue and will address it in a future update.

7.29 Unused Code

Informational **Version 1** **Code Partially Corrected**

CS-CURVE-FXSWAP-V3-024

The following described code is not used in the project. It is recommended to remove it to improve code clarity and maintainability.

1. The event `RemoveLiquidityOne` in the `Twocrypto` contract.
2. `MIN_A` and `MAX_A` constants in `StableswapMath`.
3. `price_oracle()` and `get_virtual_price()` functions defined in the interface `Curve` in `TwocryptoView`.
4. `newton_y()` in the interface `Math` in `TwocryptoView`, which is also not defined in the `StableswapMath` contract.
5. `A_MULTIPLIER` in `TwocryptoFactory`.
6. The function `_D_from_xcp()` in `Twocrypto`.

As of **Version 2**:

1. In `TwoCryptoView`, `_calc_dtoken_nofee` returns `amountsp`, but it is never used.

As of **Version 9**:

1. In `TwocryptoView`, the internal `_fee()` function is no longer used.
 2. In `TwocryptoPolicy`, the internal `_unpack_2()` function is not used.
-

Code partially corrected:

All the above code has been removed or is now used, except for:

- the `A_MULTIPLIER` in `TwocryptoFactory`
- the findings introduced in **Version 2**

- `_unpack_2` in `TwocryptoPolicy`

7.30 Unused Return Values

Informational

Version 1

Acknowledged

CS-CURVE-FXSWAP-V3-025

The system contains functions that return values that are not used by the caller.

- `Twocrypto.mint()`
- `Twocrypto.burnFrom()`

Acknowledged:

Client has acknowledged this issue and will address it in a future update.

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 Donations Shares Available to Burn Considerations

Note Version 7

Version 7 updates how `last_donation_release_ts` is set during `tweak_price`. Instead of always setting it to the current timestamp after burning donation shares, it picks a new anchor so that immediately after the burn the available donation shares equal what was available just before minus what was burned. This preserves the exact leftover (no over-reset, no extra drip).

Invariant: `_donation_shares()` = `_donation_shares()` - `donation_shares_to_burn`

Where `_donation_shares()` is the result of `_donation_shares()` after the price tweak.

Protection is a throttle in `[0, PRECISION]`: higher means slower release, lower means faster. Because availability depends on protection, the new anchor moves accordingly:

To achieve the invariant, The following should hold:

`_donation_shares(False)` = `_donation_shares(False)` - `donation_shares_to_burn`
* `_donation_shares(False)/_donation_shares()`

This means that, if a strong protection is active at the time of the burn:

1. `_donation_shares(False)` will be relatively small compared to `_donation_shares()`.
2. Shares available to burn are small compared to total shares.

On the other side, if a weak protection is active at the time of the burn:

1. `_donation_shares(False)` will be relatively large compared to `_donation_shares()`.
2. Shares available to burn are large compared to total shares.

In other words takeaways:

- After a rebalance with strong protection, expect slower subsequent dripping (smaller immediate availability).
- After a rebalance with weak protection, expect faster subsequent dripping (larger immediate availability).

8.2 Integration Considerations

Note Version 1

Integrators should be aware of the following when integrating with the system:

1. The virtual price can rise sharply, fall sharply, naturally, or with a malicious intent.
2. The same follows for the LP price returned by `lp_price()`.
3. `last_prices` is the spot price of the pool, which can be easily manipulated by a malicious actor and should not be used as a reliable indicator of the pool's price.

4. `xcp_profit` has no clear meaning, it is a derivative value used to maintain the rebalancing condition, and account for various profits, but it is not a direct profit value. It can also be smaller than 10^{18} .
5. Donations are capped at a given percentage of the pool's total supply, this means that it is possible that a donation reverts, integrators should handle this case gracefully, see [Donations can revert while released shares remain unburned](#) and [Dust donations can revert after the pre-fee mint check](#) for more details.
6. The price oracle provided by the pool implements an exponential moving average (EMA) with a period that can be adjusted by the pool's owner. It should be noted that although much more robust than `last_prices`, or a simple average, it can always be manipulated by a malicious actor willing to lose some funds for a later gain.
7. When the pool is uninitialized (no shares minted), view functions can return inconsistent values or might revert. Integrators should not rely on these functions until the pool is properly initialized.
8. View functions provide estimates but not necessarily guarantee a precise output.

8.3 Policy Callbacks Execute Under the Pool's Reentrancy Lock

Note **Version 9**

Custom Policy contracts must be aware that pool callbacks execute while the pool's `@nonreentrant` lock is held. If `Policy.update_pool_state()` calls back into a `@nonreentrant` function on the same pool, the callback will revert.

This includes the read-only wrappers `lp_price()`, `get_virtual_price()`, `price_oracle()`, and `price_scale()`. When the pool calls `update_pool_state()` with `must_succeed=True`, the revert propagates and reverts the whole transaction. In the `balanced_remove_liquidity()` path, where `must_succeed=False`, the failed policy callback is silently ignored.

8.4 View and Math Contract Addresses

Note **Version 1**

In the constructor of `Twocrypto`, both `self.VIEW` and `self.MATH` are initialized with `empty(address)`:

```
# These immutable assignments are source-patched into the blueprint at
# deploy time for deployments that embed periphery addresses.
VIEW = Views(empty(address))
MATH = Math(empty(address))
```

For the pool to be usable, the source code must be modified to replace the `empty(address)` with the actual addresses of the view and math contracts before deployment of the blueprint contract.